

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Grado en Ingeniería y Sistemas de Datos

TRABAJO DE FIN DE GRADO

Título: Aplicación de técnicas de Machine Learning para la identificación y optimización de patrones en 'Swing Chart' en mercados financieros

Autor: D. Lijun Zhou

Supervisión técnica:

Supervisión académica: Dr. Eduardo López Gonzalo

Departamento: Departamento de Señales, Sistemas y Radiocomunicaciones.

MIEMBROS DEL TRIBUNAL

Presidente:

Vocal:

Secretario:

Suplente:

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

.....

Fecha de lectura:

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Resumen

El presente proyecto tiene como objetivo el desarrollo e implementación de un sistema basado en técnicas de Machine Learning orientado al análisis y detección de patrones en “Swing Charts”, una herramienta ampliamente utilizada en el análisis técnico de los mercados financieros. A partir de datos históricos y en tiempo real, el sistema generará representaciones de “Swing Charts” a partir de “Bar Charts” y aplicará algoritmos de aprendizaje supervisado y no supervisado para identificar configuraciones recurrentes, como dobles máximos, dobles mínimos o rupturas de tendencia.

El desarrollo se llevará a cabo principalmente en Python, utilizando librerías como pandas, scikit-learn y matplotlib, además de entornos de conexión con plataformas de trading como Interactive Brokers API (IBKR). El proyecto busca evaluar la capacidad predictiva de los modelos entrenados y su utilidad en la generación de señales de compra o venta dentro de una estrategia de trading algorítmico. Con ello, se pretende aportar un enfoque innovador que combine la solidez del análisis técnico clásico con la adaptabilidad de las técnicas modernas de inteligencia artificial.

Palabras clave

Trading algorítmico, predicción de mercados financieros, automatización, análisis en tiempo real, backtesting.

Summary

XXXXXXXXX

Keywords

XXXXXXXXX

Índice

1	Introducción y objetivos	1
1.1	Introducción	1
1.2	Objetivos	2
2	Estado del arte	5
2.1	Tendencias en el trading algorítmico	5
2.2	Modelos para series temporales: RNN, LSTM y GRU	5
2.2.1	LSTM	5
2.2.2	GRU	6
2.3	Tendencias recientes en el trading cuantitativo	7
2.4	Comparativa entre indicadores tradicionales, ML clásico, ML profundo y IA agéntica	7
3	Desarrollo	9
3.1	Pipeline	9
3.2	Desarrollo del autómata	11
3.2.1	Obtención de datos de los futuros	12
3.2.2	Carga y limpieza	13
3.2.3	Cálculo de características	16

3.2.4	Selección de características	17
3.2.5	Definición y cálculo de la clase objetivo	18
3.2.6	Aprendizaje supervisado	19
3.2.7	Backtest de las predicciones	23
3.2.8	Simulación de Monte Carlo	26
3.2.9	Simulación de Monte Carlo	26
3.2.10	Gestión de cartera	27
3.2.11	Manejo del riesgo	28
3.2.12	Operación	28
3.2.13	Logging	30
3.3	Despliegue usando contenedores de Docker	31
3.3.1	Configuración del contenedor principal	31
3.3.2	Bases de datos	32
3.3.3	Visualización	34
3.3.4	Alertas y avisos	35
4	Resultados	36
4.1	Resultados entrenamiento	36
4.2	Resultados backtest y Monte Carlo individual	37
4.3	Resultados backtest y Monte Carlo de cartera en su conjunto	39
4.3.1	Comparativa del sistema contra Robotrader	41
5	Conclusiones y líneas futuras	43
5.1	Conclusiones	43
5.2	Líneas futuras	44

5.2.1	Aprendizaje no supervisado	44
5.2.2	Orquestación y MLOps	45
5.2.3	Pyomo	46
5.2.4	Resto de mejoras	47
6	Aspectos éticos, económicos, sociales y ambientales	51
6.1	Introducción	51
6.2	Descripción de impactos relevantes relacionados con el proyecto	51
6.3	Análisis detallado de alguno de los principales impactos	51
6.4	Conclusiones	51
7	Presupuesto económico	52
	Bibliografía	53
	Anexos	54
A	Resultados de los entrenamientos	54
B	Resultados de los backtest individuales	54
C	Repositorio de GitHub con el código	58

Listado de figuras

2.1	Arquitectura LSTM, [1]	6
2.2	Arquitectura GRU, [1]	6
3.1	Información estática y dinámica del contrato de futuros AD obtenida mediante la API de Interactive Brokers.	13
3.2	Comparación antes y después de ajustar	15
3.3	Ejemplo de registro JSON generado por el sistema para cada operación ejecutada.	30
3.4	Ejemplo de un <i>Dashboard</i> en Grafana	35
3.5	Ejemplo con alertas recibidas en Slack	35
4.1	Comparativa de resultados obtenidos por los modelos LSTM (fila superior) y GRU (fila inferior).	37
4.2	Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada exigente	38
4.3	Ejemplo de Monte Carlo para la mejor estrategia	39
4.4	Comparativa entre simulaciones Monte Carlo y curvas de capital con y sin límites de riesgo y valor nocional.	40
4.5	Comparativa entre simulaciones Monte Carlo y curvas de capital con y sin límites de riesgo y valor nocional.	42

7.1	Comparativa de métricas de clasificación y curvas ROC para los modelos LSTM y GRU aplicados al activo AD.	55
7.2	Comparativa de métricas de regresión y curvas de pérdida para los modelos LSTM y GRU aplicados al activo AD.	56
7.3	Comparativa de métricas de clasificación y curvas ROC para los modelos LSTM y GRU aplicados al activo ZS.	57
7.4	Comparativa de métricas de regresión y curvas de pérdida para los modelos LSTM y GRU aplicados al activo ZS.	58
7.5	Ejemplo de curva de capital operando tanto LONG como SHORT para el activo CL	59
7.6	Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada por defecto para el activo CL	59
7.7	Ejemplo de curva de capital operando tanto LONG como SHORT con un umbral de entrada exigente para el activo CL	60
7.8	Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada exigente para el activo CL	60
7.9	Ejemplo de Monte Carlo para la mejor estrategia del activo CL	61
7.10	Ejemplo de curva de capital operando tanto LONG como SHORT para el activo ZS	61
7.11	Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada por defecto para el activo ZS	62
7.12	Ejemplo de curva de capital operando tanto LONG como SHORT con un umbral de entrada exigente para el activo ZS	62
7.13	Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada exigente para el activo ZS	63
7.14	Ejemplo de Monte Carlo para la mejor estrategia del activo ZS	63

Listado de tablas

1.1	Características principales de los contratos de futuros analizados	4
3.1	Ejemplo de registro histórico OHLCV de un contrato de futuros	14
3.2	Criterios de selección de predicciones válidas en la Fase 1 del backtest.	24
3.3	Modalidades operativas utilizadas en la Fase 2 del backtest.	25
3.4	Estructura de la tabla <code>metrics</code>	32
3.5	Estructura de la tabla <code>risk_events</code>	32
3.6	Estructura de la tabla <code>signals</code>	33
3.7	Estructura de la tabla <code>trades</code>	33

Todo list

Glosario

OHLCV - Precios Open High Low y Close y Volumen

XXXX - XXXXXXXXX

Introducción y objetivos

1.1 Introducción

El trading financiero tradicional presenta una limitación fundamental, una fuerte dependencia de los criterios subjetivos, los sesgos cognitivos y el estado emocional de cada inversor. Factores como el miedo o la euforia pueden conducir a decisiones inconsistentes y alejadas de una gestión óptima del riesgo. Para mitigar este problema surgió el *trading algorítmico*, un enfoque donde las decisiones se toman en base a reglas matemáticas predefinidas, eliminando en gran medida la intervención humana durante la operativa.

La evolución natural del *trading algorítmico* ha sido la incorporación de modelos de aprendizaje automático (ML) que se entrenan usando un conjunto de características generadas a partir de datos históricos de los diferentes activos financieros a operar. Estos modelos permiten aprender patrones complejos a partir de datos históricos, capturar relaciones no lineales y adaptarse a diferentes regímenes de mercado. El presente trabajo se centra en esta línea, el diseño y desarrollo de un sistema de trading basado en ML capaz de operar de forma autónoma sobre un conjunto diverso de futuros financieros.

Para la elaboración de este trabajo se toman como referencia dos obras. Por un lado, *Trading Algorítmico con Python* de Isaac Trullàs [2], que proporciona una guía sobre el flujo de trabajo completo para el desarrollo de una estrategia algorítmica, desde la carga y limpieza de los datos, pasando por el cálculo y validación de las características que alimentarán a los modelos hasta la evaluación final mediante el cálculo de métricas para comprobar que la estrategia desarrollada es consistente, robusta y capaz de generar beneficios de forma sostenida. Por otro lado, *Safety in the Market* de David E. Bowden[3], que introduce conceptos fundamentales de análisis de mercado y del trading.

En los mercados financieros existen diferentes tipos de activos, cada uno con sus características, con diferentes niveles de riesgo y con comportamientos distintos frente a las condiciones económicas. Entre los más comunes se encuentran las acciones, que representan participaciones dentro de una empresa; los bonos, que constituyen instrumento de deuda emitidos por gobiernos o corporaciones; las materias primas, como el oro, petróleo o productos agrícolas; y las divisas, cuyo valor se basa en el tipo de cambio que existe entre monedas. Por otra parte existen derivados financieros, cuyo valor depende de un activo subyacente. Dentro de esta categoría se encuentran los futuros, contratos estandarizados que permiten tomar posiciones largas o cortas sobre índices bursátiles, divisas, materias primas o renta fija. Una posición larga significa comprar barato esperando que el precio aumente para vender, mientras que una posición corta consiste en vender anticipadamente esperando que el precio disminuya para recomprar más barato. Existe una asimetría en el riesgo asociado a cada tipo de posición, la

ventaja de las posiciones a largo reside en que en caso de que el precio disminuya, solo puede caer hasta 0 mientras que en las posiciones cortas el precio puede aumentar sin un techo, haciendo especialmente relevante la gestión de riesgo.

En este trabajo se ha optado la operación sobre futuros en vez de sobre otros activos por diversas razones.

- **Mercados regulados y centralizados:** Las negociaciones se realizan en mercados regulados y centralizados (CME, MEFF, entre otros), garantizando transparencia y control, disminuyendo los riesgos de manipulación.
- **Alta liquidez:** Ofrecen una elevada liquidez, facilitando la ejecución eficiente de órdenes incluso en estrategias algorítmicas.
- **Estandarización de los contratos:** Todos los contratos comparten especificaciones fijas —tamaño, tick, vencimiento, márgenes— lo que simplifica la obtención, comparación y tratamiento de los datos.
- **Uso de márgenes y apalancamiento:** El uso de márgenes permite operar con apalancamiento, aumentando el potencial de beneficio aunque también el riesgo. Por ello, la gestión del riesgo es fundamental, como se detallará más adelante. Este mecanismo permite controlar posiciones mayores con un capital limitado.
- **Rollover:** Existe la posibilidad de realizar *rollover*, evitando la entrega física del activo y permitiendo mantener la operativa de forma continua.

1.2 Objetivos

El objetivo principal de este presente trabajo es desarrollar un sistema capaz de operar de manera autónoma una cartera con un valor de un millón de dólares, maximizando el beneficio mientras asume el menor riesgo posible. Como se puede observar en la Tabla 1.1, el sistema operará sobre 13 futuros distintos que pertenecen a distintas categorías, divisas, materias primas, índices bursátiles y bonos del Tesoro estadounidense, proporcionando así una exposición diversificada a diferentes clases de activos y dinámicas de mercado.

A continuación se describen los futuros sobre los que se llevará a cabo la operativa.

AD - Australian Dollar

Futuro sobre el dólar australiano frente al dólar estadounidense. Muy utilizado en estrategias macro y ligadas a materias primas.

BP - British Pound

Futuro sobre la libra esterlina. Alta liquidez y relevancia en estrategias FX y cobertura frente a decisiones del Banco de Inglaterra.

CL - Crude Oil WTI

Futuro del petróleo WTI. Uno de los contratos más negociados del mundo, con elevada volatilidad y fuerte sensibilidad geopolítica.

EC - Euro FX

Futuro del euro frente al dólar. El contrato FX más líquido del CME, adecuado para estrategias direccionales y de reversión.

ES - E-mini S&P 500

Futuro del índice S&P 500 en formato reducido. Altísima liquidez y representatividad del mercado estadounidense.

GC - Gold

Futuro del oro. Activo refugio con dinámicas propias en periodos de incertidumbre y movimientos de tipos reales.

MFXI - Micro Euro FX (M6E)

Versión micro del futuro del euro. Permite exposición granular con menor margen, ideal para calibración fina.

NG - Natural Gas

Futuro del gas natural. Muy volátil y con fuerte estacionalidad, relevante en estrategias de energía.

NQ - E-mini Nasdaq 100

Futuro del Nasdaq 100. Más volátil que el ES, con fuerte exposición al sector tecnológico.

YM - E-mini Dow Jones

Futuro del Dow Jones Industrial Average. Menor volatilidad y comportamiento más estable que NQ y ES.

ZB - 30Y US Treasury Bond

Futuro del bono del Tesoro a 30 años. Muy sensible a expectativas macroeconómicas de largo plazo.

ZN - 10Y US Treasury Note

Futuro del Treasury a 10 años. El contrato de renta fija más líquido del mundo, referencia clave en estrategias macro.

ZS - Soybean Futures

Futuro de la soja. Presenta estacionalidad marcada y sensibilidad a factores climáticos y agrícolas globales.

Tabla 1.1: Características principales de los contratos de futuros analizados

Símbolo	Nombre	Mult.	Tick Size	Tick Value	Margen Init
AD	Australian Dollar	100000	0.00005	\$5	\$2503.59
BP	British Pound	62500	0.0001	\$6.25	\$2585.50
CL	Crude Oil WTI	1000	0.01	\$10	\$28336.53
EC	Euro FX	125000	0.00005	\$6.25	\$3671.13
ES	E-mini S&P 500	50	0.25	\$12.5	\$32380.88
GC	Gold	100	0.1	\$10	\$44136.61
MXFI	Micro Euro FX (M6E)	12500	0.0001	\$1.25	\$367.60
NG	Natural Gas	10000	0.001	\$10	\$6592.86
NQ	E-mini Nasdaq 100	20	0.25	\$5	\$60368.33
YM	E-mini Dow Jones	5	1	\$5	\$17296.56
ZB	30Y US Treasury Bond	1000	0.03125	\$31.25	\$4259.66
ZN	10Y US Treasury Note	1000	0.015625	\$15.625	\$2156.85
ZS	Soybean Futures	5000	0.0025	\$12.5	\$4077.58

Para alcanzar este objetivo se ha diseñado una arquitectura basada en modelos de aprendizaje automático que asisten en la toma de decisiones operativas. La propuesta inicial contempla el uso combinado de modelos de aprendizaje supervisado y no supervisado, cuyos papeles se detallará en los siguientes capítulos. En términos generales, los modelos de aprendizaje profundo se emplearán como modelos supervisados para la generación de señales de trading, mientras que los modelos no supervisados se utilizarán para identificar regímenes de mercado y agrupar comportamientos similares mediante técnicas de clustering.

La arquitectura propuesta se concibe, por tanto, como un sistema híbrido: los modelos supervisados proporcionan señales direccionales, y los modelos no supervisados aportan información contextual que puede mejorar la robustez y la interpretación del entorno de mercado.

No obstante, aunque esta sea la intención inicial, la implementación final puede diferir debido a limitaciones encontradas durante el desarrollo, cuyas motivaciones serán explicadas en los capítulos correspondientes. En cualquier caso, la arquitectura mantiene la capacidad de integrar modelos no supervisados en futuras ampliaciones.

Finalmente, el objetivo último de este trabajo es poder poner el uso la mayor cantidad de conocimiento obtenido de estos años de estudio en un solo trabajo, incluyendo principalmente programación en Python, despliegue del trabajo en Docker, entrenamiento de modelos de Machine Learning, optimización y diseño de pipelines de datos.

Estado del arte

2.1 Tendencias en el trading algorítmico

El trading algorítmico ha experimentado una evolución significativa en las últimas décadas, pasando de estrategias basadas exclusivamente en indicadores técnicos a sistemas que integran técnicas avanzadas de aprendizaje automático y modelos de aprendizaje profundo. El estado del arte actual combina ingeniería de datos, ingeniería de características, modelado predictivo, análisis cuantitativo y técnicas de optimización, con el objetivo de capturar patrones complejos en series temporales financieras altamente ruidosas y no estacionarias.

2.2 Modelos para series temporales: RNN, LSTM y GRU

Las Redes Neuronales Recurrentes (RNN) clásicas [4] fueron diseñadas para modelar dependencias temporales, pero presentan el problema del desvanecimiento del gradiente. Durante la retropropagación, los gradientes se atenúan al multiplicarse repetidamente por derivadas de funciones de activación como *sigmoid* o *tanh*, lo que provoca que las capas iniciales reciban gradientes casi nulos y solo se aprendan dependencias de corto plazo.

Para resolver esta limitación se propusieron dos arquitecturas mejoradas: LSTM (Long Short-Term Memory) [5] y GRU (Gated Recurrent Unit) [6]. Ambas introducen mecanismos de compuertas que regulan el flujo de información y permiten capturar dependencias de largo plazo de manera estable.

2.2.1 LSTM

Las LSTM introducen una memoria interna (*cell state*) y tres compuertas (entrada, olvido y salida) que controlan qué información se mantiene y cuál se descarta, permitiendo modelar dinámicas temporales complejas.

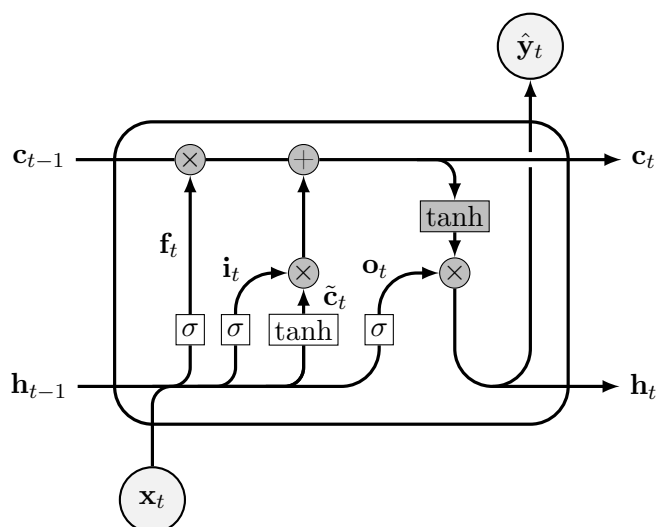


Figura 2.1: Arquitectura LSTM, [1]

2.2.2 GRU

Las GRU simplifican esta estructura mediante dos compuertas (actualización y reinicio), reduciendo el número de parámetros y acelerando el entrenamiento, manteniendo un rendimiento comparable al de las LSTM.

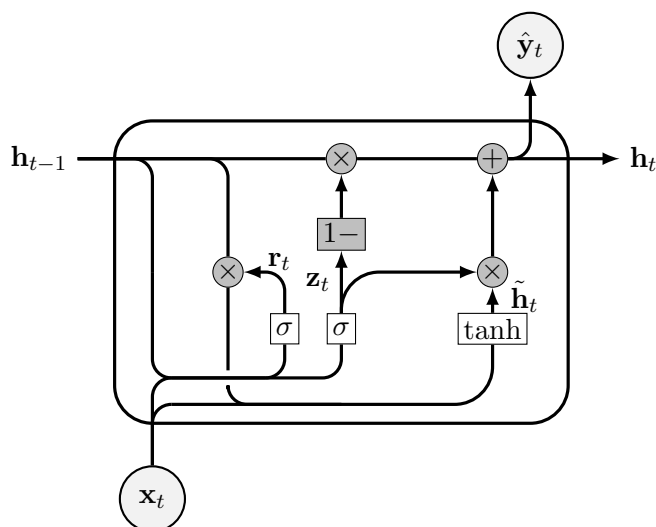


Figura 2.2: Arquitectura GRU, [1]

Ambas arquitecturas han demostrado ser especialmente útiles en series temporales financieras

debido a su capacidad para modelar dinámicas no lineales y dependencias temporales complejas y patrones ruidosos característicos de los mercados.

A pesar de su capacidad para capturar relaciones no lineales y dependencias temporales complejas, los modelos de aprendizaje profundo presentan un problema fundamental: su falta de interpretabilidad. Estas arquitecturas actúan como cajas negras, ya que no ofrecen una explicación clara de los factores que conducen a una predicción concreta. Esta opacidad dificulta comprender qué patrones del mercado está utilizando el modelo.

2.3 Tendencias recientes en el trading cuantitativo

Las tendencias recientes en el trading cuantitativo incluyen

- **Modelos basados en Transformers** para series temporales, capaces de capturar dependencias largas mediante mecanismos de atención.
- **Meta-learning** y técnicas de adaptación rápida a nuevos activos o regímenes de mercado.
- **Modelos híbridos** que combinan indicadores tradicionales con aprendizaje profundo.
- **Detección de regímenes de mercado** mediante clustering no supervisado para contextualizar las señales de trading.
- **Estudios de ablation** para evaluar la contribución real de cada componente del sistema y evitar sobreajuste.

2.4 Comparativa entre indicadores tradicionales, ML clásico, ML profundo y IA agéntica

Los enfoques utilizados en trading cuantitativo han evolucionado de manera significativa en las últimas décadas. Tradicionalmente, las estrategias se basaban en indicadores técnicos, como medias móviles, RSI, MACD o ATR, que transforman el precio en señales determinísticas. Aunque estos indicadores son sencillos de interpretar y computacionalmente eficientes, presentan limitaciones importantes: dependen de parámetros fijos, no capturan relaciones no lineales y suelen fallar en entornos de mercado cambiantes o altamente volátiles.

El siguiente paso en esta evolución fue la incorporación de modelos de aprendizaje automático clásico, como árboles de decisión, Random Forest, SVM o regresiones penalizadas. Estos modelos permiten aprender relaciones más complejas entre múltiples variables y generalizan mejor

que los indicadores tradicionales. Sin embargo, siguen teniendo dificultades para capturar dependencias temporales profundas y requieren un diseño manual de características (feature engineering) para representar adecuadamente la dinámica del mercado.

Con el auge del cómputo acelerado y la disponibilidad de grandes volúmenes de datos, surgieron los modelos de aprendizaje profundo, especialmente arquitecturas como LSTM, GRU, CNN 1D y, más recientemente, Transformers. Estos modelos son capaces de aprender patrones no lineales, dependencias de largo plazo y estructuras temporales complejas directamente a partir de los datos, reduciendo la necesidad de ingeniería manual. Su principal desventaja es la falta de interpretabilidad, ya que funcionan como cajas negras, y su tendencia al sobreajuste si no se aplican técnicas adecuadas de regularización y validación temporal.

Finalmente, en los últimos años ha emergido el concepto de IA agentica aplicada al trading, donde agentes autónomos combinan modelos predictivos, razonamiento secuencial y toma de decisiones basada en objetivos. Aunque este enfoque es prometedor, su uso en entornos financieros reales es todavía limitado debido a la complejidad de su validación, la dificultad para garantizar estabilidad y la necesidad de mecanismos robustos de control del riesgo.

Desarrollo

3.1 Pipeline

El desarrollo del sistema se estructura en varias fases dentro del proceso ETL (Extract, Transform, Load). En primer lugar, se realiza la carga de los datos históricos de los distintos contratos de futuros que se van a operar. A continuación, estos datos se someten a un proceso de limpieza y filtrado para eliminar valores erróneos, inconsistencias y posibles fuentes de ruido. Posteriormente, se lleva a cabo la transformación de los datos en un conjunto de características que servirán como entrada para los modelos de aprendizaje automático. Una vez entrenados los modelos, se evalúa su rendimiento mediante diferentes métricas y utilizando datos históricos no vistos para comprobar su capacidad de generalización. Finalmente, el sistema entra en fase operativa, donde procesa datos en tiempo real y genera señales de trading de manera autónoma.

El flujo de trabajo se organiza siguiendo la arquitectura medallón, donde los datos atraviesan por tres capas diferenciadas:

Bronce :

Contiene los datos históricos y en vivo en su forma cruda, tal y como se obtienen de las fuentes originales.

Plata :

En esta capa los datos se limpian, se validan, se filtran y se normalizan, garantizando su coherencia y calidad antes de ser utilizados.

Oro :

Los datos se transforman en *features* y *targets* listos para el entrenamiento de los modelos supervisados y para la inferencia en tiempo real.

A continuación muestro un mapa conceptual de las diferentes etapas y los hitos más importantes que se van a desarrollar en este trabajo.

1. **Carga y estandarización de datos:** descarga de datos minuto a minuto y unificación temporal a UTC.
2. **Limpieza y validación:** eliminación de datos erróneos, filtrado de fines de semana y verificación de coherencia.
3. **Enriquecimiento temporal:** asignación del día de negociación y resamplado a intervalos de 4 horas.

4. **Cálculo de variables derivadas:** generación de datos acumulados y transformaciones básicas.
5. **Ingeniería de características:** construcción de *features* avanzadas y análisis para evitar *data leakage*.
6. **Selección de características:** filtrado manual y mediante modelos tradicionales con mayor explicabilidad.
7. **Definición y cálculo de los *targets*:** construcción de las variables objetivo para los modelos supervisados.
8. **Entrenamiento y evaluación:** ajuste de hiperparámetros, búsqueda de umbrales óptimos y evaluación con métricas de clasificación, regresión y métricas financieras.
9. **Preparación para despliegue:** guardado de modelos, *scalers*, configuración y lista final de *features*.
10. **Obtención de información actualizada de los futuros:** recopilación de márgenes iniciales y de mantenimiento, multiplicadores, *tick size*, *tick value*, horarios de negociación, spread estimado y slippage medio.
11. **Backtesting y simulación:** ejecución de backtests sobre datos históricos y simulaciones Monte Carlo para evaluar robustez y sensibilidad a escenarios adversos.
12. **Operativa y monitorización:** ejecución del sistema, optimización de la cartera, control de riesgos, monitorización continua y análisis de rendimiento.
13. **KPI:** dockerización del sistema, integración con Prometheus, Loki y SQL para métricas y logs, visualización mediante grafana
14. **Alertas:** Vigilancia de las métricas y logs que al superar umbrales definidos envían alertas a Slack a través de Alertmanager

El sistema operará en interdía ya que este marco temporal ofrece diferentes ventajas para sistemas de aprendizaje automático. En primer lugar, los datos presentan menor ruido microestructural, lo que facilita el aprendizaje de patrones relevantes y disminuye el riesgo de *overfitting*. Además, los modelos no sufren por la alta no estacionalidad y los cambios de volatilidad que existen en los datos intradía. A nivel operativo, el trading interdía requiere menor monitorización continua de la situación, dado que se ejecutan menos operaciones y se mantiene las posiciones durante varios días o incluso semanas. Por otro lado, las estructuras de precio basadas en swings son más claras y consistentes en marcos interdía, lo que las hace especialmente adecuadas para la extracción de características y el análisis estructural del mercado.

3.2 Desarrollo del autómata

Uno de los objetivos principales del proyecto es garantizar que el sistema sea modular, reutilizable y fácilmente extensible, independientemente del activo financiero con el que se trabaje.

Para ello, se ha diseñado una arquitectura que separa claramente las responsabilidades y permite incorporar nuevos componentes sin modificar el núcleo del sistema.

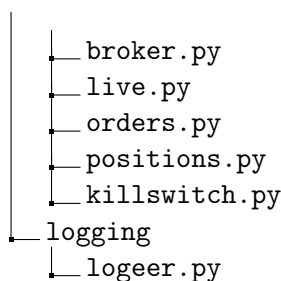
Con este propósito, la lógica del código se ha implementado dentro de un paquete instalable de Python, utilizando la instalación en modo editable mediante `pip install -e`. Esta decisión facilita el desarrollo iterativo, permite mantener una estructura coherente del proyecto y asegura que cualquier módulo pueda importarse desde otros scripts o notebooks sin necesidad de duplicar código.

La organización del paquete sigue una estructura modular que agrupa las funcionalidades por dominios lógicos, permitiendo añadir nuevos activos, modelos o pipelines de datos sin alterar la arquitectura existente.

```

/
├── paths.py
├── get_info
├── data
│   ├── loader.py
│   ├── cleaner.py
│   ├── resampler.py
│   └── to_parquet.py
├── features..... Gran cantidad de archivos para ordenar las funciones que generar las
│   ├── características
│   ├── supervised
│   │   ├── load.py
│   │   ├── models.py
│   │   ├── target.py
│   │   ├── pretrain.py
│   │   ├── train.py
│   │   ├── posttrain.py
│   │   ├── evaluate.py
│   │   └── backtest.py
│   ├── portfolio
│   │   ├── optimizer.py
│   │   └── backtest.py
│   ├── risk
│   │   ├── drawdown.py
│   │   └── spread.py
└── execution

```



El orden mostrado en la estructura anterior refleja la secuencia lógica en la que se invocan los distintos módulos del sistema. Cada directorio agrupa funcionalidades relacionadas y el flujo de ejecución avanza de forma coherente desde la carga y preparación de los datos, pasando por la generación de características y el entrenamiento de los modelos supervisados, hasta la optimización de cartera, la gestión del riesgo y la ejecución en tiempo real. Esta organización modular permite mantener un código limpio, fácilmente extensible y con responsabilidades claramente separadas.

Algunas funciones presentes en los distintos directorios del paquete son reutilizables en varias etapas del proceso, especialmente aquellas relacionadas con la obtención y carga de datos desde los archivos originales.

El código completo puede consultarse en el repositorio referenciado en el Anexo A.

A continuación se describen las distintas fases del flujo de datos y las funciones principales utilizadas en cada una de ellas.

3.2.1 Obtención de datos de los futuros

Se proporcionan los datos históricos OHLCV de distintos futuros con los que va a trabajar. Sin embargo, existen diferentes datos que no se han proporcionado, destacando el mercado donde cotiza cada futuro y la divisa con la que se negocia, estos dos datos son necesarios para posteriormente poder obtener el resto de información necesaria. Una vez identificados el mercado y la moneda de cada futuro, es posible conectarse a la API de Interactive Brokers y realizar peticiones para obtener información adicional. Esta información puede clasificarse en dos grandes grupos: **información estática** e **información dinámica**.

Dentro de la información estática o casi estática ya que no rara vez cambia, además de lo mencionado anteriormente se encuentra el identificador único de cada futuro, el multiplicador del futuro (apalancamiento implícito), el tamaño mínimo de movimiento (*tick size*) y el valor monetario asociado a este movimiento (*tick value*).

Por otro lado, la información dinámica cambia cada día o semana. Entre estos datos se encuentran los horarios de negociación, que dependen tanto de la bolsa como de los festivos del país correspondiente. Esta información permite determinar si un contrato puede operarse en

un momento dado.

También se dispone de información sobre las horas de mayor liquidez, aunque en este proyecto se emplearán estadísticas derivadas de los propios datos para identificar los periodos más líquidos y así reducir el *slippage*, entendido como la diferencia entre el precio esperado al enviar una orden y el precio real al que se ejecuta.

Finalmente, dentro de la información dinámica destacan dos parámetros fundamentales: el *margen inicial* y el *margen de mantenimiento*. El margen inicial es la cantidad mínima necesaria para abrir una posición, mientras que el margen de mantenimiento es el capital requerido para mantenerla abierta. En operativa interdía, este margen suele ser mayor para proteger al broker frente a posibles saltos bruscos de precio al inicio de la sesión, conocidos como *Price Gap* u *Opening Gap*.

```
{
  "AD": {
    "name": "Australian Dollar",
    "symbol": "AD",
    "ib_symbol": "AUD",
    "exchange": "CME",
    "currency": "USD",
    "approx_total_fee_per_side": 2.5,
    "min_slippage_ticks": 1,
    "max_spread_ticks": 3,
    "fallback_margin": 3000
  }
}

{
  "AD": {
    "name": "Australian dollar",
    "ib_symbol": "AUD",
    "conId": 854527687,
    "exchange": "CME",
    "multiplier": 100000.0,
    "tick_size": 5e-05,
    "tick_value": 5.0,
    "currency": "USD",
    "trading_hours": "20260620:CLOSED; ...",
    "liquid_hours": "20260620:CLOSED; ...",
    "approx_total_fee_per_side": 2.5,
    "min_slippage_ticks": 1,
    "max_spread_ticks": 3,
    "min_profit_ticks": 3.0,
    "initial_margin": 3000,
    "maint_margin": 2700.0,
    "fetched_at": "2026-06-20T14:42:04.151746+00:00"
  }
}
```

Figura 3.1: Información estática y dinámica del contrato de futuros AD obtenida mediante la API de Interactive Brokers.

3.2.2 Carga y limpieza

Se han facilitado los datos históricos OHLCV de distintos futuros con los que van a trabajar.

Tabla 3.1: Ejemplo de registro histórico OHLCV de un contrato de futuros

Ticker	Per	Fecha	Hora	Open	High	Low	Close	Volumen	OpenInt
AD	I	20010502	060100	0.52000	0.52000	0.52000	0.52000	2	0

Como se puede observar en la Tabla 3.1 la información facilitada incluye datos sobre el activo representado, la fecha y la hora de registro, los datos OHLCV y el interés inicial de cada día, este último dato no será utilizado para simplificar la complejidad y porque al ser un valor no relativo o porcentual no es tan recomendado para los modelos de aprendizaje profundo, además de que el interés puede cambiar a lo largo del día.

En un primer análisis se observa que las marcas temporales de los datos no están estandarizadas al formato UTC, por lo que este es el primer aspecto que debe corregirse. Para identificar la zona horaria original de cada conjunto de datos, se utilizan como referencia los horarios de apertura y cierre de cada futuro. Agrupando el volumen por horas y analizando los periodos con actividad reducida o nula, es posible detectar de forma fiable las horas en las que el mercado permanece cerrado y, a partir de ello, inferir la zona horaria utilizada en los datos históricos.

La Figura 3.2 muestra un ejemplo del procedimiento utilizado para identificar la zona horaria original de los datos. En el caso del futuro del oro (GC), puede observarse que existe una pausa diaria en el volumen entre las 23:00 h y las 24:00 h, correspondiente al cierre habitual del mercado CME Globex. Esta ventana de inactividad permite inferir que las marcas temporales del dataset están expresadas en horario de Chicago, lo que facilita su conversión posterior a UTC.

Este proceso fue uno de los pocos que no pudo automatizarse completamente, ya que requiere interpretar patrones específicos de cada mercado. No obstante, el comportamiento es suficientemente consistente como para generalizarlo: los futuros negociados en el CME utilizan habitualmente la zona horaria de Chicago (UTC-6), mientras que los futuros europeos suelen emplear la zona horaria de Europa Central (UTC+1). A partir de esta información es posible estandarizar todos los datos al formato UTC de manera fiable.

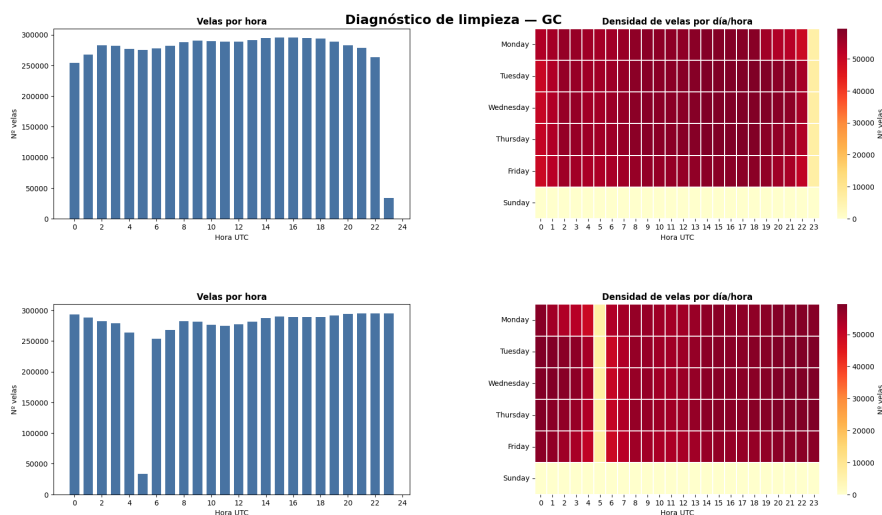


Figura 3.2: Comparación antes y después de ajustar

Después del análisis preliminar, el primer paso consiste en eliminar aquellos registros que claramente no deberían aparecer en la serie temporal, en particular los correspondientes a fines de semana, ya que los futuros no cotizan durante esos periodos y cualquier dato presente suele deberse a errores del proveedor o desajustes en los timestamps.

Los datos en horas de descanso si representan actividad real aunque sea mínima, pueden ser órdenes que llegan tarde, que se cruzan en el libro de órdenes aunque el mercado esté cerrado, actividad residual del Globex, ajustes del Exchange, ticks de apertura o cierre del libro. Además eliminarlos elimina la continuidad temporal, introduce gaps artificiales y puede afectar a diferentes métricas.

Una vez estandarizadas las marcas temporales, se incorpora una primera columna derivada que identifica el día de trading. Aunque esta variable no se utilizará directamente como entrada para los modelos, resulta útil para calcular agregados diarios que posteriormente servirán para construir otras características más avanzadas.

Con el objetivo de mejorar el rendimiento y reducir el tamaño de los archivos, los datos originalmente almacenados en formato CSV se convierten a formato Parquet. Además de ofrecer una compresión más eficiente y una lectura significativamente más rápida, Parquet permite organizar los datos en particiones independientes. Esto facilita cargar únicamente los segmentos necesarios, por ejemplo, por año o por mees, optimizando tanto el tiempo de acceso como el uso de memoria durante las fases de entrenamiento y análisis.

3.2.3 Cálculo de características

Una vez con los datos OHLCV limpios, se procede a generar el conjunto de características derivadas que servirán como entrada para los modelos de aprendizaje automático. Para ello se emplean funciones procedentes de distintas librerías.

Las características generadas abarcan múltiples categorías: de momento, estructurales, temporales, de tendencia, de volatilidad, de volumen y otras derivadas de combinaciones entre ellas. Dado que la estrategia es de tipo multidía, se incorporan además *swing charts* y un conjunto de variables construidas a partir de su estructura

Antes de calcular estas variables, los datos originales, registrados con una resolución de un minuto, se agregan en velas de 4 horas. Esta frecuencia reduce el ruido propio de las señales de muy corto plazo y resulta adecuada para una estrategia interdía, en la que las posiciones pueden permanecer abiertas durante varios días.

Para mantener una organización clara y facilitar la extensibilidad del sistema, las características se agrupan en subcarpetas según su tipología. En aquellos casos donde existe un número elevado de funciones, se realiza una subdivisión adicional para mantener una estructura modular y fácilmente mantenible.

derived

Características derivadas de otras series básicas. Incluye transformaciones estadísticas, normalizaciones, estandarizaciones y combinaciones no triviales de indicadores primarios.

momentum

Indicadores de momento y aceleración del precio. Contiene osciladores, ROC (Rate of Change) y medidas de fuerza relativa basadas en variaciones porcentuales.

returns

Cálculo de retornos simples, logarítmicos y acumulados. Incluye funciones para medir variaciones relativas entre barras y construir series de rendimiento.

selection

Módulo destinado a la selección de características. Puede incluir filtros, ranking de importancia, eliminación de colinealidad o criterios de calidad.

structural

Características que describen la estructura interna de las velas y del microcomportamiento del precio. Incluye patrones de velas, proporciones OHLC y medidas de microestructura.

swings

Detección de swings, pivotes y estructuras de máximos/mínimos relevantes. Útil para identificar cambios de tendencia y puntos de giro.

temporal

Variables temporales: hora, minuto, día de la semana, estacionalidades, ciclos intradía y cualquier codificación temporal relevante.

trend

Indicadores de tendencia: medias móviles, MACD, ADX, fuerza de tendencia y medidas de persistencia direccional.

volatility

Medidas de volatilidad histórica y realizada. Incluye ATR, bandas de Bollinger, rangos, dispersión de precios y estimadores de volatilidad intradía.

volume

Características basadas en volumen: flujo de órdenes, volumen relativo, volumen por precio, desequilibrio comprador/vendedor y estadísticas derivadas del volumen.

Una vez generadas todas las características, se realiza una primera validación para identificar la presencia de valores nulos en cada una de ellas. En este análisis destacan especialmente los swings, que por su propia naturaleza presentan numerosos huecos: solo se actualizan cuando aparece un nuevo máximo o mínimo significativo. En estos casos se opta por aplicar un forward fill, propagando el último valor válido disponible, ya que esta aproximación mantiene la coherencia estructural del indicador sin introducir información futura.

Algunas características requieren un número mínimo de velas para poder calcularse, por ejemplo, indicadores que necesitan 12 o 14 observaciones iniciales, lo que genera un pequeño bloque de filas incompletas al comienzo de la serie. Estas primeras filas se eliminan sin impacto relevante, ya que representan una fracción insignificante del total de datos disponibles. Debido a esta dependencia de ventanas largas, no resulta viable reiniciar los indicadores al comienzo de cada día, una idea inicialmente considerada. Además, dado que la estrategia es multidía, reiniciar los cálculos diarios tampoco tendría sentido operativo, pues rompería la continuidad temporal necesaria para capturar movimientos prolongados.

En una primera versión del proyecto se incluyeron únicamente las características propuestas en [2]. Sin embargo, con el objetivo de enriquecer la representación del mercado y mejorar la capacidad predictiva de los modelos, se decidió ampliar el conjunto incorporando características adicionales de mayor complejidad, incluyendo indicadores estructurales, relaciones entre activos y métricas derivadas de swing charts.

3.2.4 Selección de características

Una vez calculadas todas las características, es necesario reducir su número, ya que muchas de ellas pueden estar altamente correlacionadas o aportar información redundante. Para ello se aplican varias técnicas complementarias.

El primer paso consiste en eliminar las características con alta correlación entre sí, evitando así que el modelo reciba señales duplicadas que puedan inducir sobreajuste. También se descartan aquellas variables que, por su definición, podrían introducir data leakage, así como las características no cíclicas o no estacionarias que podrían confundir al modelo, por ejemplo, valores que aumentan con el tiempo y que podrían ser interpretados erróneamente como señales predictivas.

Posteriormente se aplica un filtrado mediante el algoritmo LASSO, utilizando un modelo clásico de scikit-learn que ofrece una mayor interpretabilidad. Este método permite identificar las características con mayor peso explicativo y seleccionar aproximadamente el 50 % más relevante, reduciendo de forma significativa la dimensionalidad del conjunto de datos.

En conjunto, estos pasos ayudan a mitigar la maldición de la dimensionalidad. Aunque este fenómeno afecta especialmente a los modelos clásicos de aprendizaje automático, también resulta beneficioso en arquitecturas más complejas, ya que reduce el riesgo de sobreajuste y disminuye los tiempos de entrenamiento.

3.2.5 Definición y cálculo de la clase objetivo

Para este proyecto se ha decidido usar el método de la *triple barrera*. Este enfoque define tres posibles eventos que pueden ocurrir tras la apertura de una operación: la activación del *Stop Loss* (SL), la activación del *Take Profit* (TP) o el vencimiento de una barrera temporal. A partir de un horizonte fijo, se determina cuál de las tres barreras es alcanzada primero. Si el precio toca el nivel de pérdida predefinido, se clasifica como SL; si alcanza el nivel de beneficio, como TP; y si no se produce ninguno de los dos movimientos dentro del tiempo establecido, se asigna la clase correspondiente a la barrera temporal. En el caso de que en una misma vela se alcancen simultáneamente SL y TP, se adopta un criterio conservador asignando el peor de los dos escenarios, es decir, SL. Los umbrales de SL y TP se definen como múltiplos del ATR, lo que permite adaptarlos a la volatilidad del activo.

Durante la fase inicial del proyecto se consideró la posibilidad de predecir directamente patrones de *swings*. Sin embargo, estos presentan varias limitaciones, entre ellas la necesidad de velas de confirmación (habitualmente dos), lo que introduce un retraso operativo significativo, en este caso, aproximadamente ocho horas. Por este motivo se descartó su uso como objetivo de predicción, aunque sí se mantienen como características relevantes para el entrenamiento de los modelos.

Además de la etiqueta discreta derivada de la triple barrera, se calcula también el retorno asociado al movimiento, concretamente el retorno logarítmico. Esta información resulta útil para la capa de regresión descrita en la siguiente sección y permite evaluar con mayor precisión la calidad de las señales: no es equivalente un acierto con un retorno del 0.2 % que uno del 2 %. El retorno logarítmico se emplea por sus propiedades matemáticas: aditividad tempo-

ral, simetría, aproximación a la normalidad y comparabilidad entre activos con volatilidades distintas.

En términos generales, el espacio de posibles objetivos es amplio: etiquetas binarias (sube/baja), ternarias (SL/TP/tiempo), regresión de retornos, clasificación de magnitud del movimiento, predicción de swings o incluso estimación de probabilidad de tocar cada barrera. Cada uno de estos enfoques implica distintos requisitos de modelado y diferentes riesgos de *data leakage*.

3.2.6 Aprendizaje supervisado

Llegados al núcleo del proyecto, el siguiente paso consiste en definir las arquitecturas de los modelos de aprendizaje supervisado que se van a emplear. La arquitectura se resume en un sistema de dos modelos paralelos con tres capas secuenciales y acoplados mediante un mecanismo de soft-voting que combina las predicciones de cada etapa de clasificación.

Las dos primeras etapas son de clasificación binaria y la tercera de regresión. Ambos modelos están definidos usando PyTorch. Cada uno de los modelos seleccionados tiene una función, LSTM está orientado a capturar dependencias a largo plazo y patrones más suaves en la secuencia, mientras que GRU tiende a reaccionar mejor a cambios rápidos, transiciones de régimen y estructuras más volátiles.

En una primera versión con solo las dos fases de clasificación se opta por realizar una votación equitativa, y si la media supera un umbral indicado para cada fase se avanza a la siguiente. Posteriormente se incorporó la tercera capa de regresión para dinamizar la votación, usando el retorno esperado como ponderación. De este modo, si uno de los modelos estima un retorno significativamente mayor, su predicción adquiere más peso en la decisión final. El retorno se calcula a un horizonte de 12 velas.

Inicialmente se consideró complementar estos modelos con algoritmos clásicos como XGBoost o Random Forest, capaces de capturar relaciones no temporales. Sin embargo, se decidió centrar el estudio en arquitecturas profundas recurrentes, más adecuadas para

3.2.6.1 Arquitectura de los modelos

Los tres modelos LSTM comparten la misma estructura general:

- **Capa recurrente:** LSTM con 2 capas, 128 unidades ocultas, `batch_first=True` y `dropout=0.3`.
- **Extracción del estado:** se utiliza el último *hidden state* h_t de la última capa.
- **Cabeza densa:**

- Lineal($128 \rightarrow 64$), GELU, Dropout(0.3)
- Lineal($64 \rightarrow 32$), GELU, Dropout(0.2)
- Lineal($32 \rightarrow \text{salida}$)

Las salidas dependen del nivel:

- **L1:** Clasificación binaria (HOLD / ACCIÓN)
- **L2:** Clasificación binaria (SELL / BUY)
- **L3:** Regresión (retorno logarítmico)

Los modelos GRU son análogos a los LSTM, sustituyendo la capa recurrente por una GRU con la misma configuración (2 capas, 128 unidades, dropout).

3.2.6.2 Motivación de las dos fases de clasificación

La decisión de utilizar dos etapas de clasificación se debe al fuerte desbalanceo presente en el objetivo generado mediante la triple barrera. Aproximadamente el 60 % de las velas terminan tocando la barrera del tiempo y mientras que las otras dos barreras solo representan un 20 % cada una. En un inicio se probaron modelos con salida ternaria, pero incluso ajustando los pesos de las clases, los resultados no fueron satisfactorios.

Por ello se optó por dividir el problema en dos fases binarias:

1. **L1:** ¿Existe movimiento significativo? (¿toca SL o TP?)
2. **L2:** En caso afirmativo, ¿en qué dirección ocurre el movimiento?

En ambas fases se evalúa la probabilidad predicha y se compara con un umbral. Aunque inicialmente se utilizaron valores fijos, finalmente ambos umbrales se seleccionaron mediante un proceso de grid search diseñado para maximizar el *F1-score* en cada nivel. También se evaluó la posibilidad de decidir la clase únicamente por diferencia relativa entre probabilidades, pero el uso de un umbral explícito, optimizado para cada fase, proporcionó una mayor estabilidad y un mejor equilibrio entre precisión y recall.

Antes de pasar al entrenamiento es necesario calcular los hiperparámetros idóneos para cada entrenamiento teniendo en cuenta el tamaño de cada dataset, el periodo de muestreo y el número de velas a futuro a predecir. Se obtienen así los siguiente hiperparámetros.

lookback

Número de velas hacia atrás que se pasa a cada momento temporal

train_size

Tamaño del bloque de entrenamiento para cada iteración

test_size

Tamaño del bloque de prueba para cada iteración

gap

Número de velas de separación entre el final de una iteración y el comienzo de la siguiente.
Sirve para evitar filtrado de datos

3.2.6.3 Entrenamiento de los modelos

El entrenamiento de los modelos se realiza de forma independiente para cada arquitectura por separado, entrenando las tres fases juntas. Para ello, se emplea un sistema de *walk forward* con *moving window*, es la técnica estándar para entrenamiento con series temporales. A diferencia del entrenamiento normal este método respeta la estructura temporal de los datos y evita data leakage.

El método *walk forward* (también conocido como *rolling forecast origin*) consiste en entrenar el modelo en un bloque de inicial de datos históricos y evaluarlo en el siguiente bloque inmediatamente posterior. A continuación se avanza hacia el siguiente bloque dejando algunas velas de margen y se repite este proceso hasta completar el entrenamiento con todos los datos posibles. La variante utilizada, *moving window*, mantiene un tamaño fijo de ventana en cada iteración, a diferencia de la opción *expanding window*, donde la ventana crece progresivamente.

Este enfoque es imprescindible porque el *cross-validation* clásico no es válido en series temporales: al mezclar aleatoriamente los datos y dividirlos en *folds* sin respetar el orden temporal, permite que el modelo entrene con información futura, introduciendo un *data leakage* masivo.

Una vez definidos los hiperparámetros, se generan los datasets correspondientes a cada iteración del walk forward, incorporando el *lookback* calculado previamente. Para cada nivel del modelo se separan los datos en entrenamiento y prueba, y se aplica un *scaler* ajustado exclusivamente con los datos de entrenamiento, evitando así cualquier fuga de información. El orden de entrenamiento es secuencial: primero L1, después L2 y finalmente L3. El resultado de cada iteración es un conjunto de predicciones y probabilidades asociadas a cada nivel.

Tras completar el entrenamiento, se procede a seleccionar los umbrales óptimos para las dos primeras capas de clasificación. Dado que la tercera capa es de regresión, no dispone de probabilidad asociada. Los umbrales se determinan mediante un proceso de *grid search* orientado a maximizar el *F1-score macro*, métrica adecuada para escenarios con fuerte desbalanceo de

clases, a diferencia de *accuracy*, *precision* o *recall*, que pueden resultar engañosas en este contexto.

Los modelos entrenados se guardan en archivos `.pth`, junto con el último *scaler* utilizado en la ventana correspondiente. Esto es necesario porque el *scaler* se recalcula en cada iteración y sus parámetros (mediana, IQR, etc.) cambian con el tiempo. También se almacenan las características empleadas, para poder reproducir el filtrado durante el backtest y la operativa real, así como los parámetros del modelo, incluyendo los umbrales seleccionados.

El *scaler* elegido es **RobustScaler**, debido a que utiliza la mediana y el rango intercuartil (IQR), lo que lo hace resistente a valores atípicos y a movimientos extremos del mercado, conocidos como *cisnes negros*. Alternativas como *StandardScaler* o *MinMaxScaler* presentan desventajas: el primero es sensible a outliers y el segundo fija un rango que puede volverse inestable ante cambios de régimen.

La votación entre modelos solo se realiza si ambos superan la primera fase y coinciden en la dirección predicha en la segunda. En caso contrario, no se genera señal. La ponderación final incorpora el retorno estimado por la capa de regresión, de modo que predicciones con mayor retorno esperado tienen mayor peso.

Una posible extensión futura consiste en actualizar tanto el modelo como el *scaler* de manera periódica (por ejemplo, semanalmente), permitiendo que el sistema se adapte mejor a cambios de régimen del mercado. Esta actualización no solo implicaría volver a entrenar desde cero, sino aplicar un proceso de *fine tuning* sobre los modelos ya existentes. El *fine tuning* permite ajustar los pesos previamente aprendidos utilizando únicamente los datos más recientes, reduciendo el coste computacional y acelerando la adaptación del modelo a nuevas condiciones de volatilidad, liquidez o comportamiento direccional. De este modo, el sistema podría mantener la estabilidad de los patrones aprendidos a largo plazo mientras incorpora información fresca del mercado, logrando un equilibrio entre memoria histórica y capacidad de reacción.

3.2.6.4 Análisis del entrenamiento

Una vez finalizado el entrenamiento de los modelos, es necesario evaluar su rendimiento utilizando métricas tanto de clasificación como de regresión. El objetivo es comprobar no solo la capacidad predictiva de cada nivel, sino también la estabilidad del entrenamiento y la coherencia de las predicciones.

En el caso de las dos primeras capas, correspondientes a problemas de clasificación binaria, se emplean métricas estándar como el *classification report*, que incluye precisión, recall y F1-score por clase. Además, se calculan métricas adicionales que permiten evaluar distintos aspectos del comportamiento del modelo:

- **AUC-ROC:** mide la capacidad del modelo para separar ambas clases en distintos umbrales.
- **Curva de fiabilidad (reliability curve):** evalúa el grado de calibración de las probabilidades predichas.
- **Matriz de confusión con conteo:** permite analizar los aciertos y errores por clase.
- **Recall y F1-score por clase:** especialmente relevantes en escenarios con fuerte desbalanceo.
- **Pérdida de entrenamiento y prueba por fold:** se registra la pérdida de la última época para cada ventana del *walk forward*, lo que permite detectar sobreajuste o inestabilidad.

Para la tercera capa, dedicada a la regresión del retorno logarítmico, se emplea un conjunto de visualizaciones y métricas que permiten evaluar la calidad de las predicciones:

- **Gráfico predicción vs. valor real (scatter plot):** muestra la relación entre el retorno predicho y el observado.
- **Distribución del error:** permite analizar la simetría y dispersión de los residuos.
- **Densidad conjunta:** evalúa la concentración de predicciones en distintas zonas del espacio real-predicho.
- **Residual vs. fitted:** ayuda a detectar patrones no capturados por el modelo o heterocedasticidad.

En conjunto, estas métricas permiten obtener una visión completa del rendimiento del sistema: desde la capacidad de clasificación en las dos primeras fases hasta la calidad de la estimación del retorno en la tercera. Además, el análisis por *folds* del *walk forward* proporciona información sobre la estabilidad temporal del modelo y su capacidad para generalizar en distintos periodos del histórico.

3.2.7 Backtest de las predicciones

Una vez verificados los resultados del entrenamiento, se procede a realizar el *backtest* con el objetivo de evaluar el comportamiento del sistema en condiciones lo más cercanas posible a la operativa real. Aunque esta simulación no incorpora ciertos elementos propios del mercado, como la latencia, constituye un punto de partida sólido para analizar la robustez de las señales generadas por los modelos.

El backtest se ejecuta utilizando datos con resolución de 4 horas debido a limitaciones de la API empleada para la descarga. Sobre estos datos se prueban cuatro estrategias operativas:

3.2.7.1 Fase 1: Selección de predicciones válidas y cálculo de PnL teórico

En esta fase se generan las predicciones finales y se calcula el PnL teórico de cada una. A partir de esta información se definen cuatro criterios alternativos para decidir qué predicciones se consideran válidas:

	Long y Short	Solo Long
Todas las predicciones	Caso 1 Se aceptan todas las predicciones que superan los umbrales de L1 y L2	Caso 2 Igual que el Caso 1, pero solo se permiten operaciones long
Predicciones que superan un umbral más estricto	Caso 3 Solo se aceptan predicciones con retorno esperado $L3$ superior al percentil 70 (quantile 0.7)	Caso 4 Igual que el Caso 3, pero solo se permiten operaciones long

Tabla 3.2: Criterios de selección de predicciones válidas en la Fase 1 del backtest.

El percentil 70 se calcula como:

$$\text{threshold}_{L3} = |L3|_{0,7}$$

Con estos cuatro criterios se calcula el Sharpe Ratio teórico de cada conjunto de predicciones antes de pasar a la fase operativa.

3.2.7.2 Fase 2: Backtest operativo con cuatro modalidades

Una vez seleccionadas las predicciones válidas, se ejecuta el backtest operativo. En esta fase sí se simula la apertura y cierre de posiciones, aplicando cuatro modalidades distintas:

	Scale-in: No	Scale-in: Sí
Sizing dinámico: No	Estrategia 1 1 posición máxima tamaño fijo (1)	Estrategia 2 hasta 5 posiciones tamaño fijo (1)
Sizing dinámico: Sí	Estrategia 3 1 posición máxima tamaño variable (≤ 5)	Estrategia 4 hasta 25 posiciones tamaño variable (≤ 5)

Tabla 3.3: Modalidades operativas utilizadas en la Fase 2 del backtest.

3.2.7.3 Métricas de evaluación

Para evaluar el rendimiento se utilizan las siguientes métricas:

Max Drawdown

El *Max Drawdown* mide la mayor caída relativa del capital desde un máximo histórico hasta un mínimo posterior. Es una métrica fundamental para evaluar el riesgo, ya que cuantifica la peor racha de pérdidas acumuladas que puede experimentar la estrategia. Un drawdown elevado indica periodos prolongados de pérdidas y menor estabilidad operativa.

$$\text{MDD} = \max_t \left(\frac{\max_{s \leq t} \text{Equity}(s) - \text{Equity}(t)}{\max_{s \leq t} \text{Equity}(s)} \right)$$

Sharpe Ratio

El *Sharpe Ratio* mide el retorno ajustado por riesgo, comparando el rendimiento medio del sistema con la volatilidad de sus retornos. Un Sharpe alto indica que la estrategia genera beneficios consistentes con una variabilidad relativamente baja. En este proyecto, el Sharpe se calcula utilizando como capital inicial un valor equivalente a **10 veces el *initial margin*** del activo, lo que permite normalizar la métrica entre activos con diferentes requisitos de margen.

$$\text{Sharpe} = \frac{E[R_p]}{\sigma_p}$$

Calmar Ratio

El *Calmar Ratio* relaciona el retorno anualizado con el riesgo máximo asumido (Max Drawdown). A diferencia del Sharpe, que se basa en la volatilidad, el Calmar penaliza directamente las caídas profundas del capital. Es especialmente útil en estrategias con drawdowns prolongados o distribuciones de retornos no gaussianas.

$$\text{Calmar} = \frac{\text{Retorno anualizado}}{\text{Max Drawdown}}$$

CAGR (Compound Annual Growth Rate)

El *CAGR* representa la tasa de crecimiento anualizada del capital, suponiendo que los beneficios se reinvierten y que el crecimiento es compuesto. Es una métrica clave para comparar estrategias con diferentes duraciones, ya que resume el rendimiento total en una única tasa anual equivalente. A diferencia del retorno total, el CAGR suaviza la variabilidad temporal y ofrece una visión más estable del crecimiento a largo plazo.

$$\text{CAGR} = \left(\frac{\text{Equity}_{\text{final}}}{\text{Equity}_{\text{inicial}}} \right)^{\frac{1}{\text{años}}} - 1$$

Opcionalmente, se puede incluir un contador de rachas de victorias y derrotas para analizar la estabilidad operativa del sistema.

3.2.8 Simulación de Monte Carlo

3.2.9 Simulación de Monte Carlo

Como paso posterior al backtest se realiza una simulación de Monte Carlo con el objetivo de estimar la dispersión de las métricas de rendimiento y evaluar la robustez del sistema frente a diferentes secuencias posibles de resultados. Existen dos enfoques principales para aplicar Monte Carlo en este contexto. El primero consiste en reordenar o remuestrear las predicciones ya generadas por los modelos durante el backtest. El segundo, más complejo, implica generar nuevos datos OHLCV sintéticos a partir de estadísticas como medias, varianzas o correlaciones, y volver a alimentar los modelos para obtener predicciones completamente nuevas.

En este proyecto se opta por la primera aproximación, ya que permite obtener una estimación fiable de la variabilidad de las métricas sin necesidad de generar datos sintéticos ni volver a entrenar los modelos. Además, este método es computacionalmente más eficiente y evita introducir supuestos adicionales sobre la dinámica de precios.

La técnica utilizada es un *overlapping block bootstrap*. A diferencia del bootstrap clásico, que remuestrea observaciones individuales, el block bootstrap remuestrea bloques completos de la serie para preservar parte de la estructura temporal, como autocorrelaciones de corto plazo o patrones locales. El uso de solapamiento garantiza que todos los segmentos posibles de la serie original puedan aparecer en las simulaciones, evitando limitarse únicamente a bloques fijos y disjuntos. En este caso la idea inicial era la de emplear bloques de 24 velas, lo que equivale aproximadamente a una semana de predicciones, pero al contar con solo 5 meses de datos para el backtest y teniendo filtros bastantes restrictivos se optó al final por agrupar por días y así pasar en muchos casos de solo tener dos semanas a tener varios días con movimientos, haciendo más efectivo la simulación.

3.2.10 Gestión de cartera

Una vez entrenados los modelos para cada futuro, es necesario coordinar sus señales para evitar problemas de liquidez, sobreexposición o conflictos entre activos. Para ello, en esta versión del proyecto se emplea un método de gestión de cartera sencillo y manual, cuyo objetivo es garantizar que las operaciones se ejecutan de forma coherente con las restricciones de capital y riesgo.

El procedimiento seguido es el siguiente:

1. Se generan las predicciones para todos los futuros disponibles en el instante temporal considerado.
2. Se filtran aquellas predicciones que superan los umbrales definidos en las capas L1 y L2.
3. Las predicciones válidas se ordenan por nivel de confianza total, de mayor a menor.
4. Se recorre esta lista ordenada y, para cada predicción, se comprueba si cumple las restricciones operativas: disponibilidad de capital, límites de exposición por activo y restricciones de riesgo.
5. Si la predicción cumple las condiciones, se ejecuta la operación correspondiente; en caso contrario, se descarta y se pasa a la siguiente.

Este enfoque permite controlar la asignación de capital de manera simple y transparente, evitando situaciones en las que múltiples activos compitan por recursos limitados o generen niveles de riesgo excesivos. Aunque se trata de un método básico, resulta adecuado para una primera versión del sistema y permite evaluar el comportamiento conjunto de los modelos en un entorno multiactivo.

En esta fase se realiza además un *backtest* específico para la cartera completa. A diferencia del *backtest* individual por activo, cuyo objetivo era comprobar el funcionamiento general de los modelos y la coherencia de las señales, este *backtest* es más restrictivo, ya que incorpora todas las limitaciones reales de operativa además de márgenes iniciales y de mantenimiento que ya se tenían en cuenta antes: capital inicial de 1 000 000 USD, valor notional por contrato y para la cartera completa, porcentaje máximo de riesgo por operación y una exposición total máxima del 25 % del capital disponible.

Del mismo modo, se ejecuta una nueva simulación de Monte Carlo a nivel de cartera, utilizando los PnL diarios resultantes de este *backtest* más estricto. La metodología es la misma que en la simulación individual, pero aplicada ahora a la serie conjunta de beneficios de toda la cartera, lo que permite analizar la dispersión de las métricas bajo un escenario multiactivo y con restricciones reales de riesgo.

La optimización formal de la cartera mediante técnicas matemáticas más avanzadas (por ejemplo, programación lineal o cuadrática) se considera una línea de trabajo futura, ya que no se ha implementado en esta versión del proyecto.

3.2.11 Manejo del riesgo

Una vez entrenados los modelos y antes de pasar a la operativa, es necesario definir el nivel de riesgo que se está dispuesto a asumir. La métrica principal utilizada en este proyecto es el *Max Drawdown*. Este valor actúa como límite superior de pérdidas aceptables: si la curva de capital alcanza dicho umbral, se cierran todas las posiciones abiertas para evitar un deterioro mayor del capital.

Además del drawdown, se establece un porcentaje máximo del equity que puede estar simultáneamente expuesto en posiciones abiertas. Este *Exposure Limit* define el capital máximo que puede quedar comprometido en operaciones activas, independientemente del apalancamiento implícito de cada instrumento. De este modo se evita que un conjunto de señales coincidentes genere un nivel de riesgo agregado excesivo.

Otro aspecto relevante es la gestión operativa en torno a la apertura y cierre de las sesiones. Durante estos periodos el *spread* suele ampliarse y la liquidez puede ser menor, lo que incrementa el riesgo de *slippage*. Para mitigar este efecto, se incorporan reglas basadas en la hora y el volumen: en la apertura se retrasa la toma de decisiones hasta 15 minutos después o hasta que el volumen supere un umbral mínimo, mientras que en el cierre se adelanta la ejecución si el volumen comienza a disminuir. Estas reglas permiten operar únicamente en condiciones de mercado más estables.

Finalmente, se realizan comprobaciones de integridad de los datos antes de alimentar los modelos: verificar que los precios *Open* y *Close* se encuentran entre *Low* y *High*, que *Low* es menor o igual que *High*, que el volumen no es negativo y que no existen duplicados. Estas validaciones son esenciales para evitar errores silenciosos que puedan afectar tanto al entrenamiento como a la operativa.

3.2.12 Operación

Una vez completado el entrenamiento de los modelos, la configuración del sistema y la definición de las reglas de riesgo, el último paso consiste en integrar todos los componentes en un único módulo operativo capaz de conectarse a la API de Interactive Brokers y ejecutar las operaciones en tiempo real.

El flujo general de la operación es el siguiente:

1. Conexión a la API de Interactive Brokers
2. Lectura de los futuros sobre los que se van a operar y obtener los contratos específicos a operar
3. Descarga de los datos históricos necesarios para realizar las predicciones iniciales
4. Carga de la información de configuración del sistema
5. Carga de los modelos, los *scalers*, y la configuración individual de cada futuro
6. Ejecución periódica de predicciones cada 4 horas
7. Activación de un *watchdog* encargado del control de riesgo y del estado del sistema.

Dado que los futuros tienen fecha de vencimiento, el sistema obtiene diariamente la lista de contratos activos para cada símbolo y los ordena por volumen con el fin de seleccionar el contrato más líquido, minimizando así el *spread*. Si existen posiciones abiertas en un contrato que deja de ser el más líquido, el comportamiento depende de la configuración: o bien se cierran las posiciones inmediatamente, o bien se compara la diferencia de volumen con un umbral. Si la diferencia es pequeña, se mantiene la posición abierta aunque las predicciones futuras se realicen sobre el nuevo contrato principal; si la diferencia supera el umbral, se cierran las posiciones y se migra al contrato más líquido.

Es posible configurar el comportamiento antes una predicción HOLD tras una predicción LONG o SHORT, existen dos configuraciones. En el primer caso, se decide seguir con la posición abierta, mientras que en el segundo caso, se cierra la posición. Algo parecido ocurre con predicciones sucesivas opuestas, se puede decidir cerrar la posición o cerrar la posición y abrir una nueva posición opuesta a la anterior.

Toda esta lógica se controla mediante un archivo de configuración en formato JSON. Además, cada operación ejecutada se almacena en un archivo JSON que registra toda la información relevante de la operación.

```
{
  "id": 1248,
  "symbol": "GC",
  "status": "closed",
  "side": "long",
  "quantity": 1,
  "timestamp_open": "2024-03-12T10:40:00Z",
  "timestamp_close": "2024-03-13T14:00:00Z",
  "entry_price": 2145.3,
  "exit_price": 2162.7,
  "stop_loss": 2138.0,
  "take_profit": 2170.0,
  "pnl": 174.0,
  "pnl_pct": 0.81,
  "return": 0.0081,
  "max_favorable_excursion": 0.0124,
  "max_adverse_excursion": -0.0048,
  "model_signal": "BUY",
  "model_confidence_l1": 0.87,
  "model_confidence_l2": 0.72,
  "model_reg_value": 0.0045,
  "thresholds_used": {
    "threshold_move": 0.55,
    "threshold_dir": 0.60
  },
  "commission": 2.5,
  "slippage": 0.0,
  "equity_before": 10234.5,
  "equity_after": 10408.5,
  "bar_index_open": 58291,
  "bar_index_close": 58347,
  "durationBars": 56,
  "duration_minutes": 1344
}
```

Figura 3.3: Ejemplo de registro JSON generado por el sistema para cada operación ejecutada.

3.2.13 Logging

El sistema incorpora un módulo de *logging* que actúa de forma transversal en todas las fases del flujo. Aunque no constituye una etapa independiente, es un componente esencial para garantizar la trazabilidad, la depuración y la supervisión del sistema.

Durante el entrenamiento se registran mensajes informativos sobre los activos procesados, los resultados obtenidos y posibles incidencias en los datos. En las fases de backtesting y simulación se almacenan métricas, configuraciones utilizadas y cualquier anomalía detectada.

El punto más crítico para el *logging* es la fase de operación. En este contexto se registran, entre otros:

- el estado de la conexión con la API,
- el contrato seleccionado para cada futuro,
- la correcta descarga y carga de datos,
- las configuraciones activas,
- las predicciones generadas,
- las órdenes enviadas y su estado,
- el número de posiciones abiertas,
- las alertas del *watchdog* de riesgo.

Este sistema de registro permite auditar cualquier operación, reproducir escenarios pasados y detectar rápidamente fallos o comportamientos inesperados.

3.3 Despliegue usando contenedores de Docker

Una vez desarrollado el autómata y verificado su funcionamiento, se ha optado por empaquetar el proyecto dentro de un contenedor Docker con el objetivo de facilitar su despliegue en otros equipos y garantizar la reproducibilidad del entorno. El uso de contenedores permite aislar las dependencias, fijar versiones concretas de los paquetes utilizados y asegurar que el sistema se ejecuta de forma idéntica independientemente de la máquina anfitriona.

3.3.1 Configuración del contenedor principal

Dado el diseño modular del sistema y por motivos de simplicidad y velocidad, todo el flujo de operación se ejecuta dentro de un único contenedor. Este contenedor incluye el código desarrollado, las dependencias necesarias y la lógica de operación.

El proceso de construcción del contenedor es sencillo: se copian los módulos del proyecto al interior de la imagen y se instalan las versiones específicas de las librerías utilizadas. De este modo se obtiene un entorno estático y controlado, evitando incompatibilidades entre versiones o diferencias entre entornos locales.

El contenedor ejecuta directamente el módulo principal de operación, encargado de la conexión con la API de Interactive Brokers, la descarga de datos, la generación de predicciones y la ejecución de órdenes. Para ello únicamente es necesario incluir en la imagen el archivo `P50peration.py` y el paquete desarrollado, ya que el entrenamiento de los modelos se realiza desde el entorno local y no forma parte del flujo operativo en tiempo

3.3.2 Bases de datos

Al ejecutar el sistema dentro de un contenedor, resulta natural incorporar también una base de datos para almacenar de forma persistente las métricas generadas durante la operación, el historial de operaciones y otros datos relevantes. Aunque inicialmente se utilizaban archivos JSON para registrar las operaciones, el uso de una base de datos relacional permite una gestión más estructurada, consultas más eficientes y una integración más sencilla con herramientas de visualización.

En este proyecto se utiliza PostgreSQL como motor de base de datos, acompañado de PgAdmin para facilitar la inspección manual de los datos. Parte de la información también puede visualizarse desde Grafana, descrito en la siguiente sección.

Dado que se almacenan distintos tipos de información, métricas, señales, riesgo, operaciones, etc, se han definido varias tablas especializadas, cada una con su propia estructura. Estas tablas se detallan a continuación:

Tabla 3.4: Estructura de la tabla `metrics`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Marca temporal de la métrica
metric_name	TEXT	Nombre de la métrica (pnl, sharpe, latency, etc.)
value	DOUBLE PRECISION	Valor numérico de la métrica
tags	JSONB	Información adicional (símbolo, timeframe, etc.)

Tabla 3.5: Estructura de la tabla `risk_events`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Momento del evento
event_type	TEXT	Tipo de evento (drawdown, latency, stale_data...)

Continúa en la siguiente página

Campo	Tipo	Descripción
severity	TEXT	Nivel (info, warn, critical)
details	JSONB	Información adicional del evento

Tabla 3.6: Estructura de la tabla `signals`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Marca temporal de la señal
symbol	TEXT	Símbolo del futuro
contract	TEXT	Contrato específico
signal_value	DOUBLE PRECISION	Valor de la señal
model_version	TEXT	Versión del modelo
features	JSONB	Features usadas en la predicción
decision	TEXT	accepted / rejected
reason	TEXT	Motivo del rechazo (spread, volumen, riesgo...)
tags	JSONB	Información adicional

Tabla 3.7: Estructura de la tabla `trades`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
symbol	TEXT	Símbolo del futuro
contract	TEXT	Contrato específico
status	TEXT	Estado de la operación
side	TEXT	Dirección
quantity	INTEGER	Número de contratos
timestamp_open	TIMESTAMPTZ	Momento de apertura
timestamp_close	TIMESTAMPTZ	Momento de cierre
entry_price	DOUBLE PRECISION	Precio de entrada
exit_price	DOUBLE PRECISION	Precio de salida
stop_loss	DOUBLE PRECISION	Stop loss asignado
take_profit	DOUBLE PRECISION	Take profit asignado
pnl	DOUBLE PRECISION	Beneficio/pérdida absoluto
pnl_pct	DOUBLE PRECISION	Beneficio/pérdida porcentual
return	DOUBLE PRECISION	Retorno relativo
max_favorable_excursion	DOUBLE PRECISION	Máximo movimiento a favor

Continúa en la siguiente página

Campo	Tipo	Descripción
max_adverse_excursion	DOUBLE PRECISION	Máximo movimiento en contra
model_signal	TEXT	Señal generada por el modelo
model_confidence_11	DOUBLE PRECISION	Confianza del modelo (nivel 1)
model_confidence_12	DOUBLE PRECISION	Confianza del modelo (nivel 2)
model_reg_value	DOUBLE PRECISION	Valor regresivo del modelo
thresholds_used	JSONB	Umbrales usados en la decisión
commission	DOUBLE PRECISION	Comisión aplicada
slippage	DOUBLE PRECISION	Slippage registrado
equity_before	DOUBLE PRECISION	Equity antes de la operación
equity_after	DOUBLE PRECISION	Equity después de la operación
bar_index_open	INTEGER	Índice de barra de apertura
bar_index_close	INTEGER	Índice de barra de cierre
durationBars	INTEGER	Duración en barras
durationMinutes	INTEGER	Duración en minutos

3.3.3 Visualización

Una vez que todo el sistema está desplegado en contenedores, se incorpora un módulo de visualización que permite monitorizar en tiempo real y en diferido toda la información generada durante la operación. Para ello se utilizan varios contenedores adicionales: *Pushgateway*, *Prometheus*, *Loki*, *Promtail* y *Grafana*.

Prometheus y Pushgateway se encargan de la gestión y recolección de métricas, mientras que Promtail y Loki gestionan los logs del sistema. Pushgateway y Promtail actúan como recolectores, y Prometheus y Loki como motores de almacenamiento y consulta. Por su parte, Grafana se conecta tanto a Prometheus como a Loki y también directamente al contenedor de PostgreSQL, lo que permite visualizar métricas, logs y datos almacenados en la base de datos desde un único panel.

Toda la información se consume después en un *dashboard* en grafana donde se agrupa toda la información.

Toda esta información se integra en un *dashboard* de Grafana donde se agrupan las métricas más relevantes. Entre ellas se incluyen indicadores de rendimiento como la *equity curve*, el *max drawdown*, el *Sharpe ratio*, la volatilidad diaria, el *profit factor*, el *win rate* y el rendimiento por futuro. También se muestran métricas operativas, logs del sistema y estadísticas de riesgo.

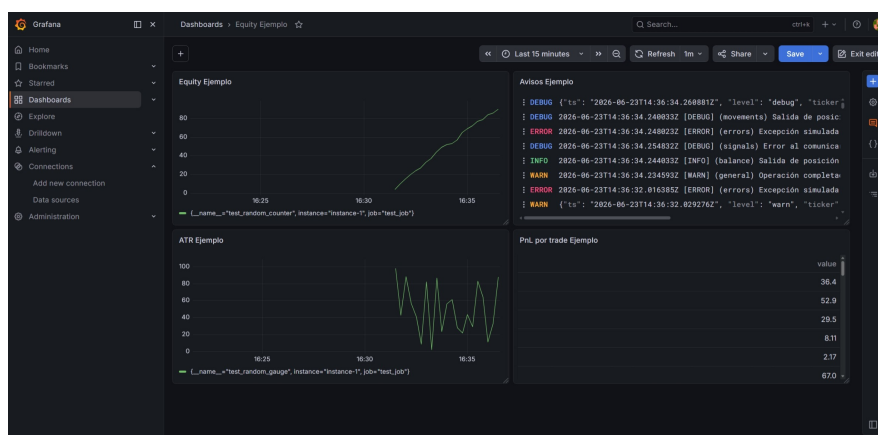


Figura 3.4: Ejemplo de un *Dashboard* en Grafana

3.3.4 Alertas y avisos

Además de la visualización en Grafana, se incorpora un sistema de alertas para recibir notificaciones ante eventos relevantes. Estas alertas se definen mediante reglas en *Alertmanager*, que forma parte del ecosistema de Prometheus. Cuando alguna de las reglas se activa, por ejemplo, un error en la conexión con la API, un aumento inesperado del riesgo, un fallo en la descarga de datos o un comportamiento anómalo del sistema, Alertmanager envía una notificación al dispositivo móvil del usuario.

En este proyecto las alertas se envían a través de Slack, utilizando una API key configurada previamente. Esto permite recibir avisos en tiempo real incluso cuando el sistema está ejecutándose de forma desatendida.

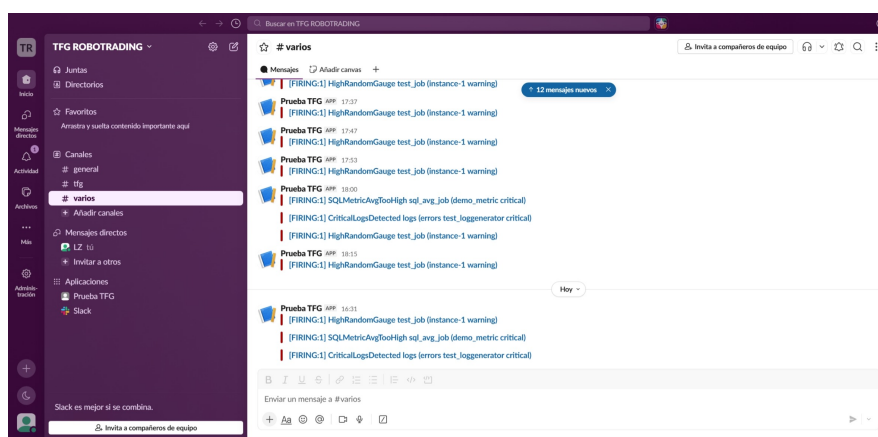


Figura 3.5: Ejemplo con alertas recibidas en Slack

Resultados

Antes de detallar los resultados obtenidos, es importante destacar que el rendimiento del sistema ha sido consistentemente positivo en todas las fases del estudio. Tanto las métricas de entrenamiento como las simulaciones de backtest y Monte Carlo, tanto a nivel individual como de cartera, muestran un comportamiento robusto y coherente, lo que permite analizar cada bloque de resultados con una base sólida.

4.1 Resultados entrenamiento

Los resultados obtenidos durante la fase de entrenamiento son, en términos generales, positivos y coherentes con el comportamiento esperado para modelos secuenciales aplicados a series temporales financieras. En primer lugar, en las métricas de clasificación se observa que la clase HOLD es sistemáticamente la que alcanza un mayor F1-score, reflejando que el modelo identifica con mayor fiabilidad los periodos sin movimiento significativo. No obstante, las clases LONG y SHORT también presentan valores de F1-score superiores a 0.33, lo que indica que, pese al desequilibrio inherente del problema, el modelo es capaz de capturar señales direccionales con un rendimiento razonable.

Además, al comparar ambos modelos, se aprecia que GRU obtiene en general mejores métricas de clasificación que LSTM, especialmente en las clases minoritarias, mostrando una mayor capacidad para discriminar entre LONG y SHORT.

El análisis de las matrices de confusión confirma esta tendencia: cuando el modelo se equivoca, la mayoría de los errores corresponden a predicciones clasificadas como HOLD, en lugar de confundir LONG con SHORT o viceversa. Este patrón es deseable en un entorno de trading, ya que reduce el riesgo de tomar posiciones en la dirección opuesta al movimiento real del mercado.

En cuanto al rendimiento probabilístico, los valores de AUC-ROC se sitúan consistentemente por encima de 0.5, con la mayoría de los futuros en el rango 0.6-0.7. La capa L2 tiende a mostrar un comportamiento ligeramente superior, lo que sugiere que la predicción de dirección (LONG vs SHORT) es más estable que la detección de movimiento (MOVE vs HOLD). Las curvas de reliability muestran un patrón similar: la calibración de L2 es más cercana a la diagonal ideal, indicando una mejor correspondencia entre probabilidad estimada y frecuencia observada.

En la parte de regresión, los errores se concentran mayoritariamente en el intervalo $[-0.02, 0.02]$, lo que implica que las predicciones de magnitud del movimiento presentan una desviación reducida respecto al valor real. En este caso, la comparación entre modelos muestra que LSTM y GRU presentan un rendimiento muy similar, sin diferencias significativas en la distribución

del error ni en la estabilidad de las predicciones.

Por último, las curvas de loss muestran una evolución estable y comparable entre folds. Aunque existen algunos casos aislados con pérdidas significativamente mayores, estos representan una fracción muy pequeña dentro de los más de cien folds totales. Además, en un entorno de trading real, estos episodios pueden mitigarse mediante mecanismos de gestión de riesgo, evitando que afecten de forma sustancial al rendimiento global del sistema.

A continuación se facilita una comparación entre ambos modelos para un mismo futuro, en el Anexo B se puede encontrar el resto de comparaciones para los demás futuros.

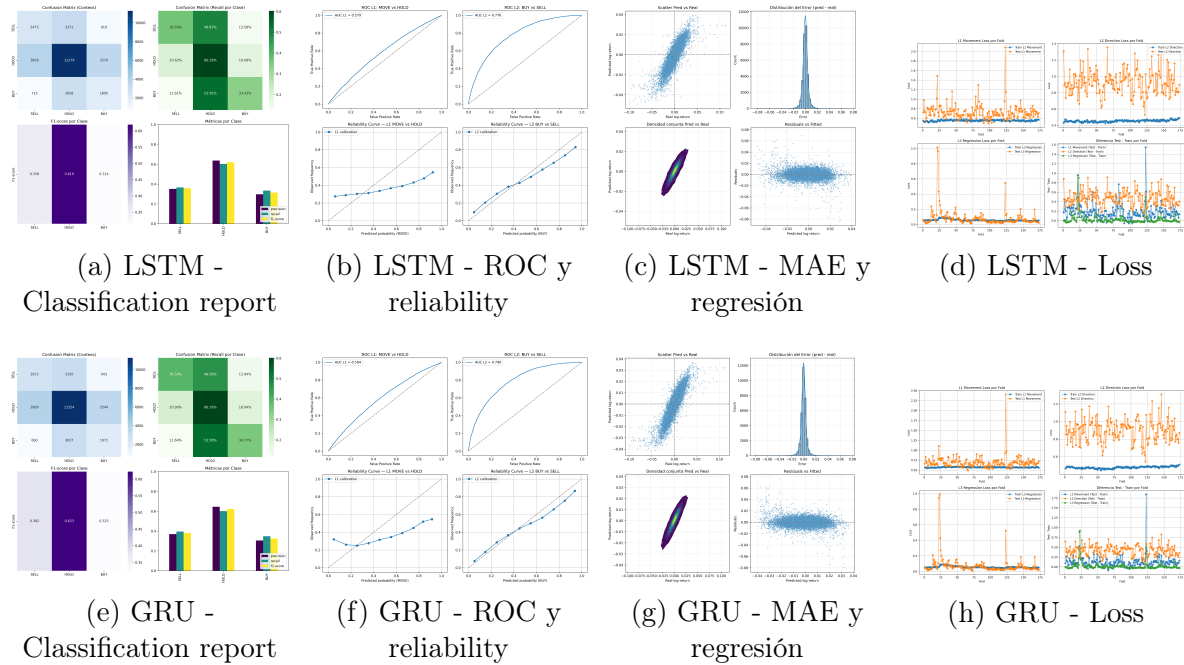


Figura 4.1: Comparativa de resultados obtenidos por los modelos LSTM (fila superior) y GRU (fila inferior).

4.2 Resultados backtest y Monte Carlo individual

Para el periodo comprendido entre finales de enero de 2026 y principios de junio de 2026, se evaluaron los trece futuros seleccionados mediante backtesting individual y simulaciones Monte Carlo. Durante este proceso, uno de los activos presentó problemas en la generación de features, por lo que no pudo ser incluido en el análisis, y otros dos no generaron señales operativas en este intervalo temporal. En consecuencia, los resultados se centran en los diez futuros que sí realizaron operaciones durante el periodo de prueba.

La primera Figura 4.2 muestra, para un tipo concreto de predicción (por ejemplo, long strong), el comportamiento de las cuatro variantes operativas.

Estas curvas permiten comparar cómo cada estrategia de ejecución afecta a la evolución del equity, al max drawdown y al Sharpe ratio. En el ejemplo mostrado, correspondiente al futuro CL, la combinación scale-in + sizing es la que obtiene el mejor rendimiento relativo frente a las otras tres variantes, mostrando una curva de capital más estable y un perfil riesgo-beneficio más favorable.

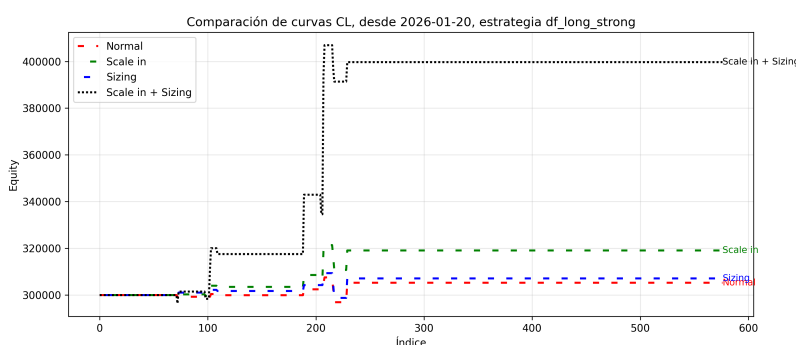


Figura 4.2: Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada exigente

La segunda Figura 4.3 recoge los resultados del Monte Carlo individual, donde se representa la curva de capital real junto con los intervalos P25-P75 y P5-P95, así como la distribución del PnL acumulado final y del porcentaje de max drawdown. Esta simulación permite evaluar la robustez del sistema frente a permutaciones aleatorias de las operaciones. En el caso mostrado, la curva real se sitúa en la parte superior de las distribuciones, lo que indica que el rendimiento observado no es un resultado extremo ni dependiente de una secuencia particularmente favorable de operaciones.

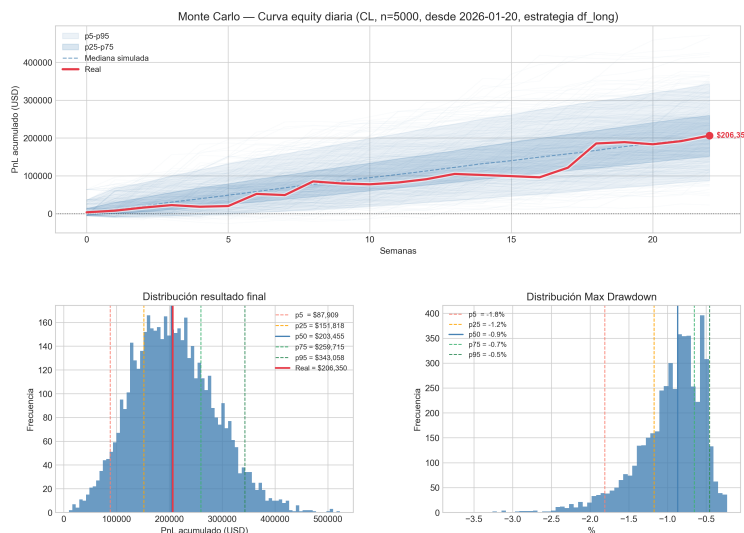


Figura 4.3: Ejemplo de Monte Carlo para la mejor estrategia

El resto de resultados se encuentra en el Anexo C

4.3 Resultados backtest y Monte Carlo de cartera en su conjunto

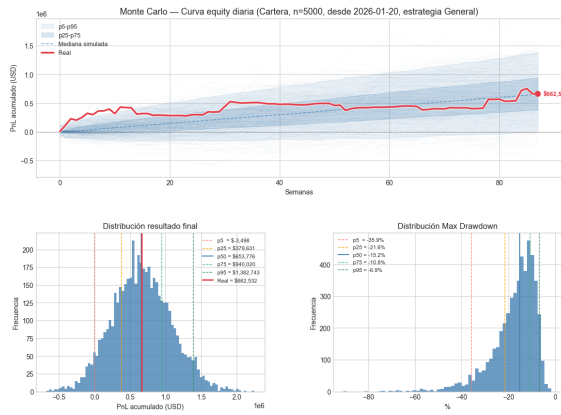
Los resultados muestran una diferencia muy significativa entre operar la cartera sin restricciones y aplicar límites realistas de riesgo y valor nocional. En el escenario sin restricciones, la curva de capital alcanza aproximadamente 660 000 USD de beneficio acumulado durante el periodo analizado. Sin embargo, al introducir límites de exposición, márgenes y valor nocional máximo por activo, el beneficio final se reduce a alrededor de 460 000 USD.

Esta diferencia se explica principalmente porque, al limitar el valor nocional y la exposición máxima, varios futuros dejan de poder operarse en determinados momentos, ya sea por superar los márgenes requeridos o por exceder el porcentaje máximo de capital permitido. En consecuencia, la cartera ejecuta menos operaciones y reduce su apalancamiento efectivo, lo que disminuye el beneficio potencial pero también controla mejor el riesgo.

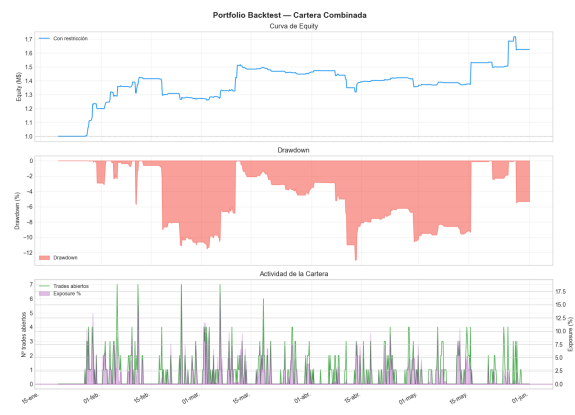
En cuanto al perfil de riesgo, la versión con restricciones presenta un máximo drawdown cercano al 12 %, un valor razonable para una cartera diversificada de futuros. El Sharpe ratio, aunque positivo, se sitúa por debajo de 2, reflejando un equilibrio más conservador entre rentabilidad

y volatilidad una vez aplicados los límites operativos.

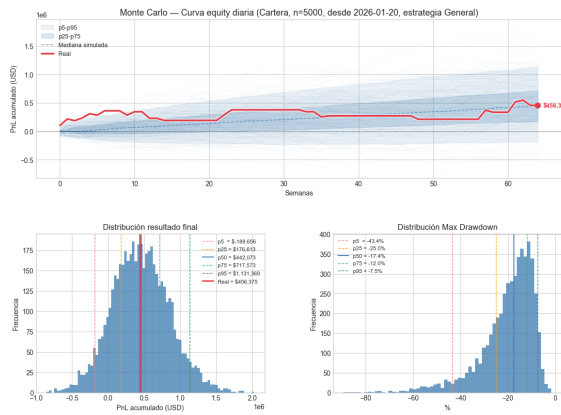
En conjunto, estos resultados muestran que la cartera completa mantiene un comportamiento robusto incluso bajo condiciones realistas de operativa profesional. Aunque la rentabilidad disminuye al aplicar límites de riesgo, la estabilidad del sistema mejora y el perfil riesgo-beneficio se vuelve más adecuado para una implementación real.



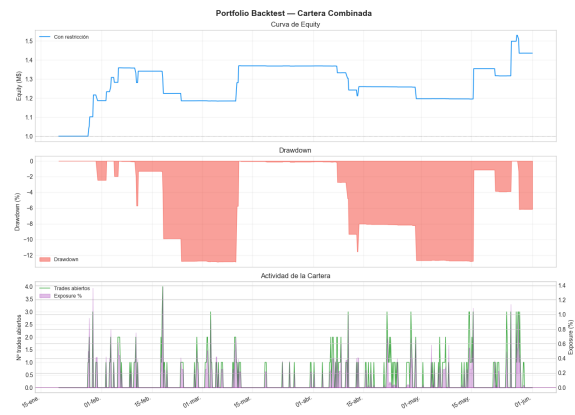
(a) Monte Carlo sin límites de riesgo ni valor notional



(b) Equity curve sin límites de riesgo ni valor notional



(c) Monte Carlo con límites de riesgo y valor notional



(d) Equity curve con límites de riesgo y valor notional

Figura 4.4: Comparativa entre simulaciones Monte Carlo y curvas de capital con y sin límites de riesgo y valor notional.

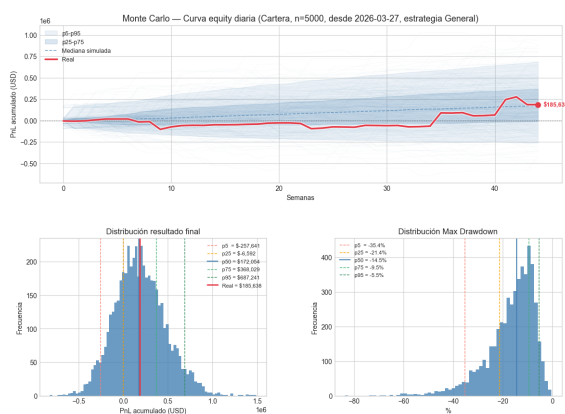
4.3.1 Comparativa del sistema contra Robotrader

Para contextualizar el rendimiento del sistema, se ha realizado una comparación directa con los resultados obtenidos por los participantes de la competición Robotrader, cuyo periodo operativo abarcó los meses de abril y mayo. Es importante destacar que estos datos no se utilizaron en ningún momento para entrenar los modelos, evitando así cualquier tipo de leakage.

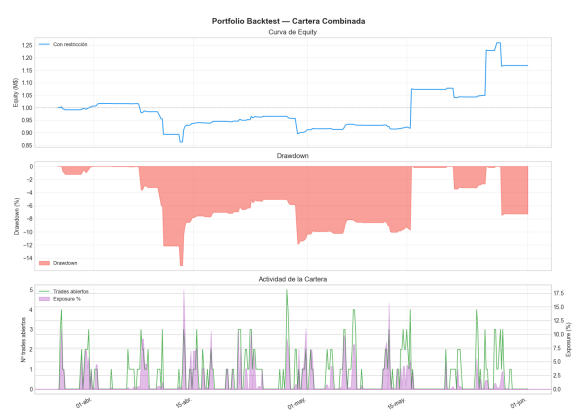
En este intervalo concreto, el sistema obtiene aproximadamente 185 000 USD de beneficio sin restricciones y alrededor de 75 000 USD al aplicar límites realistas de riesgo y valor nocional. Esta diferencia es incluso más marcada que en el periodo completo, ya que la competición se desarrolla en un entorno de elevada volatilidad y con un número reducido de operaciones, lo que amplifica el impacto del apalancamiento y de las restricciones de margen.

Si se compara únicamente el PnL final, que es la métrica más directa y comparable, el sistema se situaría aproximadamente entre las primeras posiciones de un total de 58 participantes. Además, a diferencia de una parte significativa de los competidores, el sistema no habría incurrido en pérdidas, lo que refuerza la solidez del enfoque incluso bajo condiciones exigentes.

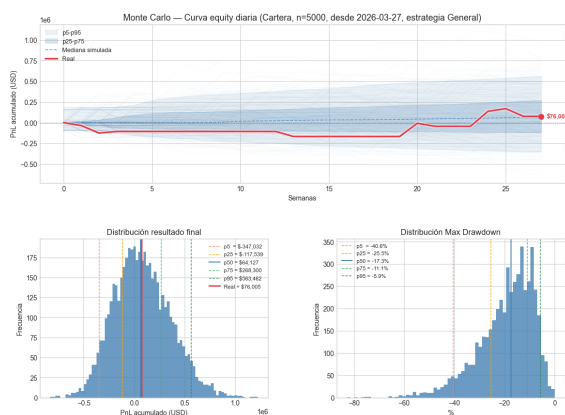
Aun así, es importante remarcar que esta comparación no es perfecta: el trading en vivo introduce factores adicionales como latencia, slippage variable, disponibilidad de liquidez y ejecución parcial de órdenes, que no pueden reproducirse completamente en un backtest. No obstante, la simulación incorpora márgenes, valor nocional y límites de exposición, lo que permite aproximarse razonablemente al comportamiento real y obtener una referencia útil del rendimiento relativo del sistema.



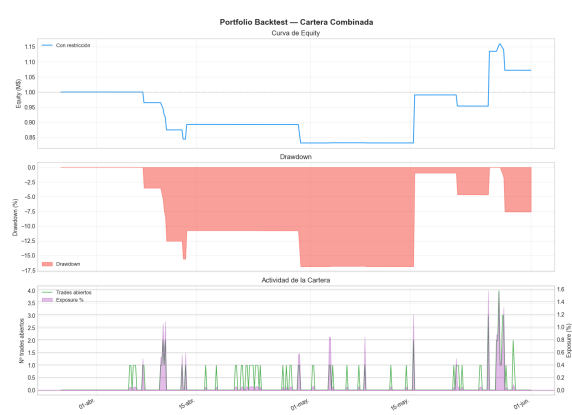
(a) Monte Carlo sin límites de riesgo ni valor nominal



(b) Equity curve sin límites de riesgo ni valor nominal



(c) Monte Carlo con límites de riesgo y valor nominal



(d) Equity curve con límites de riesgo y valor nominal

Figura 4.5: Comparativa entre simulaciones Monte Carlo y curvas de capital con y sin límites de riesgo y valor nominal.

Conclusiones y líneas futuras

5.1 Conclusiones

A lo largo de este proyecto he desarrollado un sistema completo de trading algorítmico basado en modelos secuenciales aplicados a series temporales financieras. El trabajo ha abarcado todas las fases de un pipeline profesional: desde la construcción y auditoría de features, el diseño y entrenamiento de modelos LSTM y GRU, la optimización operativa por activo, y finalmente la evaluación mediante backtesting y simulaciones de Monte Carlo tanto individuales como a nivel de cartera.

Los resultados obtenidos permiten concluir que el sistema se comporta de forma robusta, estable y coherente con lo esperado para un enfoque de predicción direccional en mercados de futuros. En la fase de entrenamiento, los modelos muestran métricas positivas, con especial solidez en la clase HOLD y un rendimiento superior del modelo GRU en clasificación, mientras que en regresión ambos modelos presentan un comportamiento muy similar. Las simulaciones individuales confirman que, para la mayoría de los activos, las estrategias optimizadas generan beneficios consistentes y mantienen un perfil de riesgo razonable.

A nivel de cartera, la diferencia entre operar con y sin restricciones de riesgo es notable: el beneficio acumulado pasa de aproximadamente 660 000 USD a 460 000 USD en el periodo completo, y de 185 000 USD a 75 000 USD en el periodo equivalente a la competición Robotrader. Aun así, incluso bajo límites estrictos de exposición, margen y valor nocional, la cartera mantiene un máximo drawdown cercano al 12 % y un Sharpe ratio positivo, lo que demuestra que el sistema es capaz de generar rendimientos atractivos sin asumir un riesgo excesivo.

Este proyecto me ha permitido aplicar de forma integrada los conocimientos adquiridos durante los cuatro años de carrera, especialmente en áreas como aprendizaje automático, análisis de datos y diseño de sistemas. El desarrollo del proyecto me ha permitido profundizar en el diseño de redes neuronales, en la gestión del riesgo y en la construcción de sistemas de trading reproducibles y evaluables.

En conjunto, el trabajo demuestra que es posible construir un sistema de trading algorítmico basado en modelos de aprendizaje profundo que sea rentable y robusto frente a diferentes escenarios de mercado. Además, abre múltiples líneas de mejora futura, tanto en la incorporación de nuevas features como en el uso de arquitecturas más avanzadas, técnicas de ensemble o integración con datos en tiempo real.

5.2 Líneas futuras

A pesar del alcance conseguido en este proyecto, existen múltiples líneas de mejora y extensiones que permitirían aumentar la robustez, escalabilidad y capacidad analítica del sistema. A continuación se presentan las principales direcciones de trabajo futuro, organizadas según su relevancia técnica y su impacto potencial.

5.2.1 Aprendizaje no supervisado

Además del uso de modelos supervisados, una de las líneas de trabajo más relevantes consiste en incorporar técnicas de aprendizaje no supervisado en distintos puntos del pipeline. Este enfoque permitiría enriquecer la información disponible para los modelos, mejorar la robustez del sistema y capturar estructuras latentes del mercado que no están explícitamente codificadas en las features actuales.

5.2.1.1 Agrupación de activos (clustering)

El primer uso previsto es la agrupación automática de los futuros en función de su comportamiento. En lugar de realizar agrupaciones arbitrarias (por ejemplo, agrupar todas las divisas o todos los índices), la idea es que el sistema determine qué activos son realmente similares. Esto permitiría:

- generar features basadas en relaciones entre activos,
- aplicar ideas inspiradas en *Pairs Trading*,
- comparar cada activo con el centroide de su cluster y el resto de clusters,
- reducir el riesgo de concentrar demasiadas operaciones en activos altamente correlacionados.

Este enfoque presenta varios retos. Por un lado, los datos históricos no siempre cubren los mismos periodos, lo que obliga a recortar o alinear las series. Por otro lado, los horarios de negociación difieren entre mercados, lo que introduce huecos que deben rellenarse mediante técnicas como forward fill. También es necesario decidir qué tipo de modelo utilizar: uno paramétrico con número fijo de clusters (como K-means) o uno no paramétrico que determine automáticamente el número óptimo (como DBSCAN o HDBSCAN). Finalmente, es crucial seleccionar adecuadamente las features sobre las que se realizará la agrupación.

5.2.1.2 Segmentación de regímenes de mercado

El segundo uso consiste en identificar regímenes de mercado mediante clustering o modelos de cambio de estado. Esta información podría utilizarse de dos maneras:

- como feature adicional para los modelos supervisados,
- o para entrenar modelos especializados en cada régimen, mejorando su precisión y estabilidad.

Aunque los modelos secuenciales ya capturan parte de esta estructura, una segmentación explícita permitiría interpretar mejor el comportamiento del sistema y adaptar la estrategia a condiciones de mercado cambiantes.

5.2.2 Orquestación y MLOps

Una vez desarrollado un sistema robusto con un autómata funcional y mecanismos de alerta tanto operativos como de seguridad, una de las líneas de mejora más relevantes consiste en incorporar herramientas de MLOps que permitan automatizar, monitorizar y versionar todo el ciclo de vida del modelo. En particular, las dos tecnologías clave para este propósito son MLflow y Apache Airflow.

5.2.2.1 MLflow: registro y gestión del ciclo de vida de modelos

MLflow es una plataforma diseñada para gestionar todas las fases del ciclo de vida de modelos de Machine Learning. Sus funcionalidades principales incluyen:

- registro completo de experimentos (métricas, hiperparámetros, artefactos),
- comparación sistemática entre modelos,
- versionado de modelos y artefactos,
- almacenamiento centralizado de resultados,
- trazabilidad total del proceso de entrenamiento.

Integrar MLflow permitiría sustituir el entrenamiento manual por un proceso estructurado y reproducible. Además, facilitaría decidir cuándo un modelo debe ser reemplazado por una versión actualizada, así como mantener un historial claro de los modelos utilizados, descartados o en fase de prueba.

5.2.2.2 Airflow: orquestación del ciclo de vida del sistema

Apache Airflow es un orquestador de flujos de trabajo que permite programar, monitorizar y ejecutar tareas complejas mediante DAGs (Directed Acyclic Graphs). Su integración con MLflow resulta especialmente útil para automatizar:

- entrenamientos semanales,
- fine-tuning incremental con datos recientes,
- evaluación automática de nuevos modelos,
- comparación contra el modelo actualmente en producción,
- promoción del mejor modelo,
- reinicios controlados del bot,
- despliegue y parada automática del sistema (por ejemplo, arrancar el sábado por la tarde y detener el viernes).

De este modo, Airflow actuaría coordinando todas las tareas necesarias para mantener el modelo actualizado y estable.

5.2.2.3 Limitaciones actuales

El principal inconveniente de Airflow es su elevado consumo de recursos, especialmente memoria RAM. Esto dificulta su uso continuo en entornos con recursos limitados. Aun así, una opción viable sería activarlo manualmente durante los fines de semana para ejecutar las tareas relacionadas con el ciclo de vida del modelo, dejando su uso permanente para una futura infraestructura más potente.

5.2.3 Pyomo

Una vez que el sistema estuvo completamente operativo, una de las mejoras naturales consistió en incorporar un módulo de optimización formal que permitiera superar algunas limitaciones del enfoque manual. En particular, el método basado únicamente en ordenar predicciones por nivel de confianza puede conducir a situaciones en las que una señal con menor confianza genere un beneficio mayor que otra con confianza superior. Para abordar este problema, se planteó el uso de Pyomo, una librería de optimización matemática en Python que permite formular y resolver problemas complejos mediante programación lineal, cuadrática o no lineal.

Pyomo ofrece un marco flexible para definir modelos de optimización de forma declarativa, especificando variables, parámetros, restricciones y funciones objetivo. En este proyecto, su uso permitiría construir un optimizador de cartera cuyo objetivo fuese maximizar el Sharpe Ratio bajo restricciones realistas de riesgo y exposición.

El proceso general para formular el problema en Pyomo seguiría los siguientes pasos:

1. **Definir el conjunto de activos** que componen la cartera.
2. **Establecer los parámetros** del modelo: retornos esperados, volatilidades, matriz de covarianza, capital disponible, límites de exposición y restricciones de riesgo.
3. **Definir las variables de decisión**, como la apertura o no de una posición, o el peso asignado a cada activo.
4. **Incorporar las restricciones** necesarias: suma de pesos, límites de exposición, márgenes, restricciones operativas y de riesgo.
5. **Formular la función objetivo**, en este caso la maximización del Sharpe Ratio.
6. **Seleccionar un solver** adecuado para resolver el problema (por ejemplo, IPOPT, GLPK o CBC).
7. **Interpretar la solución** y compararla con el método manual para evaluar mejoras en rendimiento y estabilidad.

La integración de Pyomo permitiría obtener una asignación óptima de posiciones basada en criterios matemáticos y no únicamente en la confianza de las predicciones, lo que podría mejorar la consistencia del sistema y reducir situaciones subóptimas. Además, este enfoque abre la puerta a futuras extensiones, como optimización multiobjetivo, límites por grupo de activos o modelos específicos para distintos regímenes de mercado.

5.2.4 Resto de mejoras

Además de las líneas principales descritas anteriormente, existen diversas mejoras adicionales que podrían ampliar significativamente las capacidades del sistema y su aplicabilidad práctica. Estas propuestas abarcan desde aspectos de visualización e interacción hasta la incorporación de conceptos teóricos avanzados y mejoras en la calidad de los datos.

5.2.4.1 Interfaz gráfica personalizada

Aunque Grafana ha sido útil para la monitorización inicial, una interfaz gráfica propia permitiría una personalización total del sistema. Gracias a la experiencia adquirida en asignaturas

de desarrollo Front-End, sería posible construir una aplicación que:

- muestre información más detallada que la disponible en Grafana,
- permita interactuar directamente con el bot (arrancar, detener, seleccionar activos),
- integre visualizaciones avanzadas y paneles específicos para cada módulo del sistema.

Esto proporcionaría una experiencia más completa y adaptada a las necesidades del usuario final.

5.2.4.2 Chatbot integrado (RASA o LLM)

La interfaz podría complementarse con un asistente conversacional capaz de responder preguntas sobre:

- el estado del sistema,
- métricas recientes,
- señales generadas,
- riesgos y exposición,
- decisiones operativas.

RASA sería una solución más ligera y controlable, mientras que un LLM permitiría respuestas más flexibles y contextuales. A largo plazo, podría integrarse un MCP (Model Context Protocol) para conectar el chatbot con el resto de componentes del sistema.

5.2.4.3 Incorporación de conceptos avanzados de la carrera

Existen varias ideas teóricas que podrían enriquecer el sistema y aportar nuevas perspectivas analíticas:

Entropía de Shannon y teoría de la información

Permitiría medir la incertidumbre del mercado y detectar periodos de mayor desorden o imprevisibilidad.

Cadenas de Markov y FSM

Las cadenas de Markov modelan transiciones entre estados de mercado, mientras que las máquinas de estados finitos (FSM) pueden estructurar el comportamiento del bot en función de dichos estados.

Análisis de sentimientos

El uso de modelos NLP como BERT o MarIA permitiría incorporar información textual procedente de noticias y redes sociales, complementando los indicadores cuantitativos.

5.2.4.4 Validación y calidad de datos

La integración de Great Expectations permitiría automatizar la validación de datos y detectar anomalías antes de alimentar los modelos. Esto reforzaría la fiabilidad del pipeline y reduciría errores derivados de datos corruptos o inconsistentes.

5.2.4.5 Pipeline de datos más estructurado

Durante mis prácticas pude observar el uso de pipelines altamente estructurados, donde cada paso es una función independiente que comparten un contexto común. Adoptar este enfoque permitiría:

- mejorar la modularidad,
- facilitar el mantenimiento,
- simplificar la depuración,
- y preparar el sistema para una futura orquestación completa.

5.2.4.6 Nuevos productos financieros

Aunque el proyecto se ha centrado en futuros por razones justificadas, sería interesante explorar:

- acciones,
- opciones,
- ETFs,
- o incluso criptomonedas.

Estos productos podrían utilizarse tanto como activos operables pero también como fuentes adicionales de features, enriqueciendo el conjunto de señales disponibles.

5.2.4.7 Mayor profundidad de datos

Los datos históricos utilizados son OHLCV, pero Interactive Brokers ofrece información más detallada, especialmente la separación entre precios bid y ask. Incorporar esta información permitiría:

- mejorar la precisión de las señales,
- modelar mejor el spread,
- y aproximar más fielmente las condiciones reales de mercado.

Aspectos éticos, económicos, sociales y ambientales

6.1 Introducción

Desde el punto de vista económico el sistema tiene influencia en la reducción de costes necesarios para operar una cartera de activos, permitiendo a profesionales autónomos o quienes lo toman como un hobby. En el ámbito ético y social, . Y por último, con respecto al impacto ambiental, este sistema no tiene casi impacto al poder ejecutarse en local en hardware particular no profesional.

6.2 Descripción de impactos relevantes relacionados con el proyecto

Impacto económico

1

Impacto ético y social

2

Impacto ambiental

3

6.3 Análisis detallado de alguno de los principales impactos

El impacto mayoritario de este proyecto se centra en el tema económico

6.4 Conclusiones

Como conclusión final

Presupuesto económico

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total
		360	15 €	5.400 €
COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido)	1.500,00 €	6	5	150,00 €
COSTE TOTAL DE RECURSOS MATERIALES				150,00 €
GASTOS GENERALES (costes indirectos)	15 %	sobre CD		832,50 €
BENEFICIO INDUSTRIAL	6 %	sobre CD+CI		333,00 €
SUBTOTAL PRESUPUESTO				6.715,5 €
IVA APLICABLE			21 %	1.410,255 €
TOTAL PRESUPUESTO				8.127,755 €

Bibliografía

- [1] F. Love, “Nntikz: Tikz diagrams for deep learning and neural networks,” 2024.
- [2] I. Trullàs, *Trading algorítmico con Python: Desarrolla tus habilidades en el Trading Algorítmico. Con ejemplos e integración a MetaTrader5*. Amazon Kindle Direct Publishing, 2024. Kindle edition.
- [3] D. E. Bowden, *Safety in the Market: The Smarter Starter Pack*. Queensland, Australia: Safety in the Market, third revised edition ed., 1997. ASIN: B000LYUHJE.
- [4] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [5] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.

Anexos

A Resultados de los entrenamientos

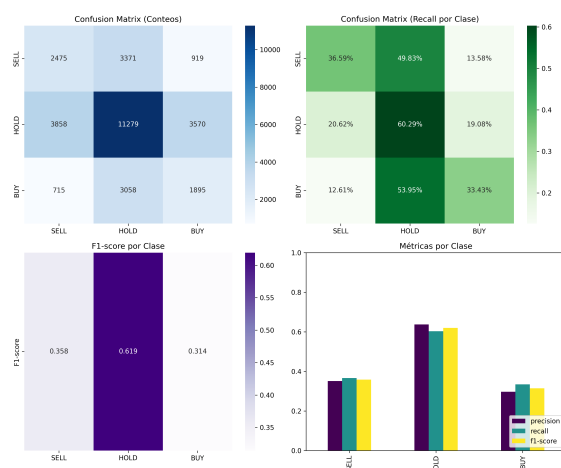
El código completo desarrollado para este trabajo se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/LijunZhou000/ML-Algorithmic-Trading>

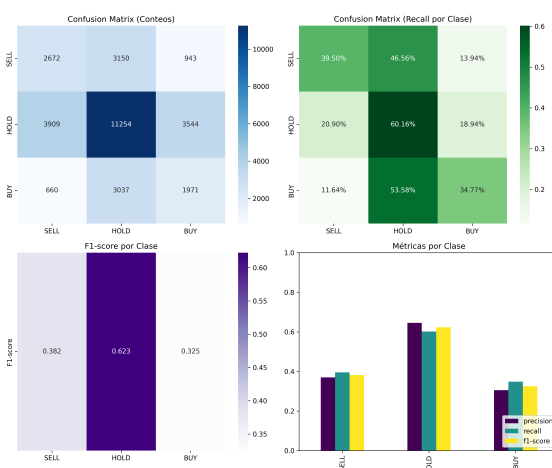
B Resultados de los backtest individuales

Debido a la gran cantidad de imágenes se ha optado por subir las imágenes a GitHub, clasificados por futuro.

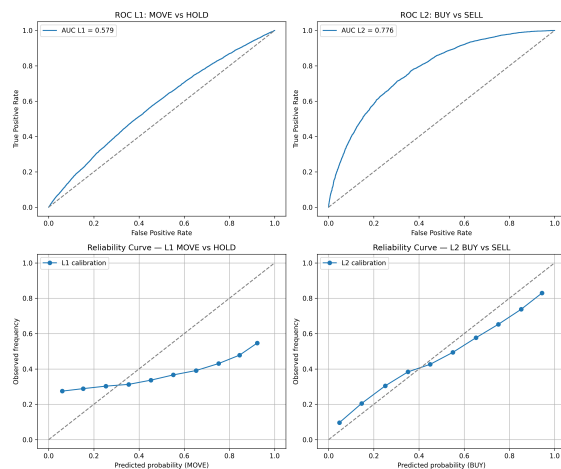
<https://github.com/LijunZhou000/ML-Algorithmic-Trading/tree/main/TFG/img/Models>



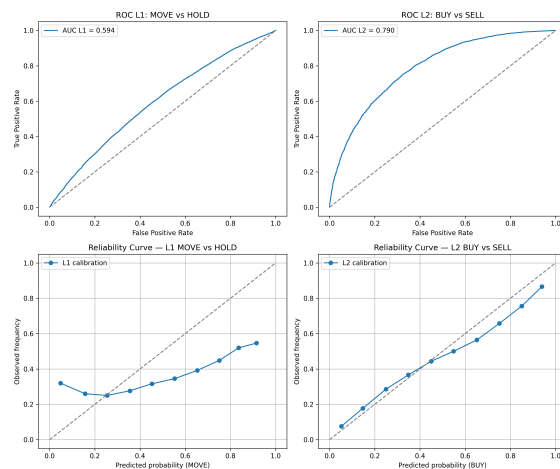
(a) LSTM - Classification report (Activo: AD)



(e) GRU - Classification report (Activo: AD)

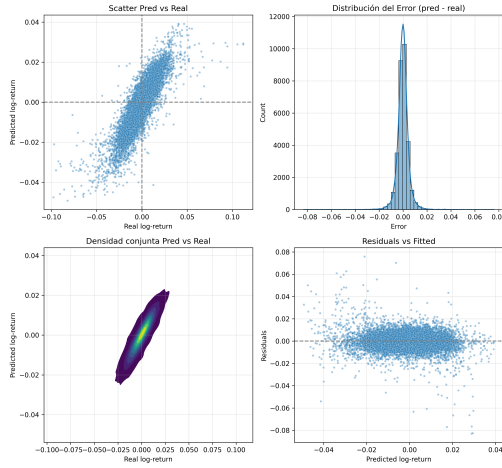


(b) LSTM - ROC y reliability (Activo: AD)

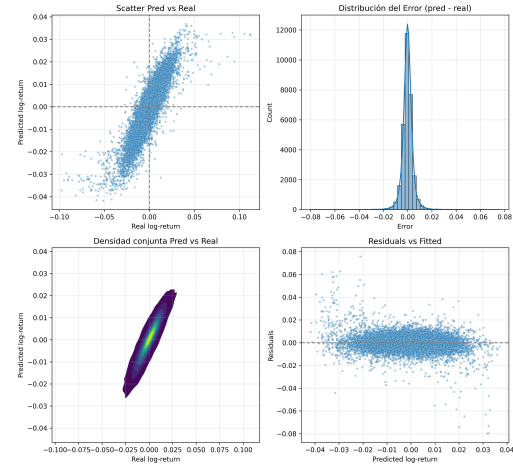


(f) GRU - ROC y reliability (Activo: AD)

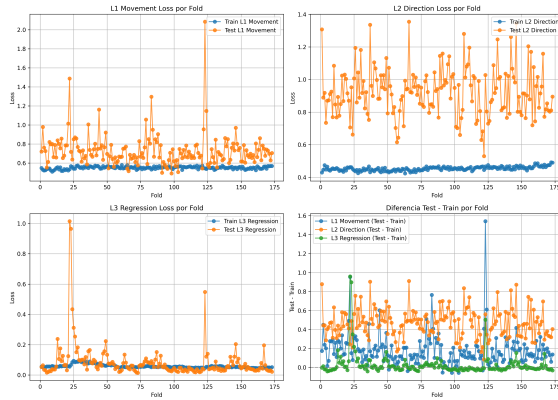
Figura 7.1: Comparativa de métricas de clasificación y curvas ROC para los modelos LSTM y GRU aplicados al activo AD.



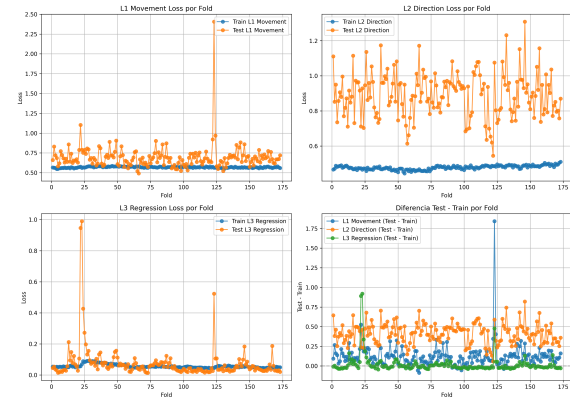
(c) LSTM - MAE y regresión (Activo: AD)



(g) GRU - MAE y regresión (Activo: AD)

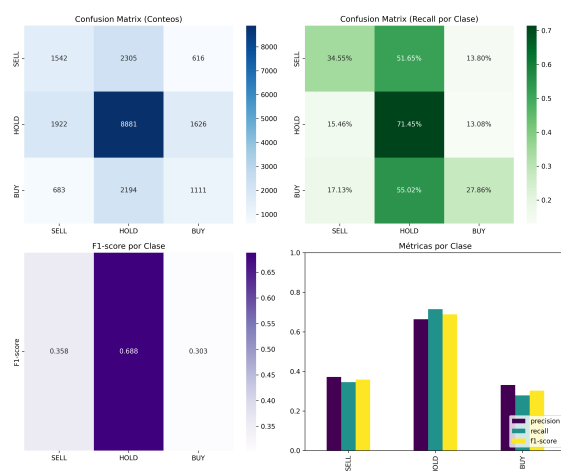


(d) LSTM - Loss (Activo: AD)

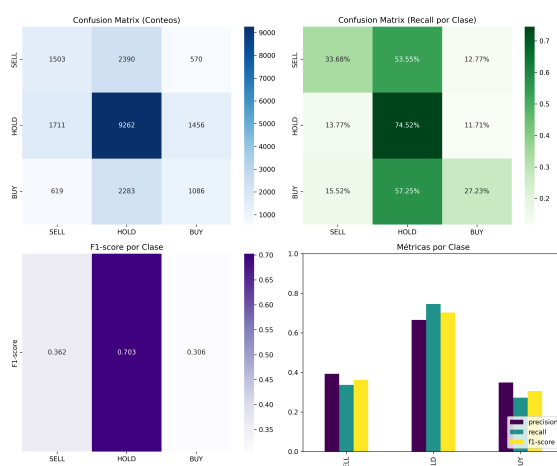


(h) GRU - Loss (Activo: AD)

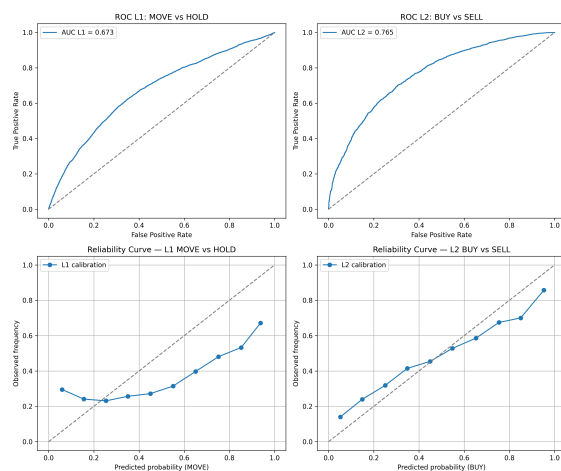
Figura 7.2: Comparativa de métricas de regresión y curvas de pérdida para los modelos LSTM y GRU aplicados al activo AD.



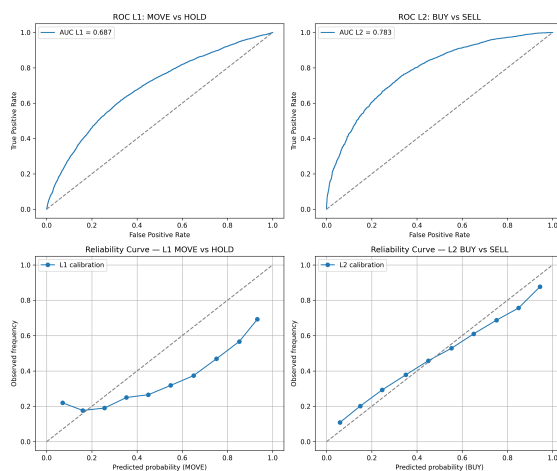
(a) LSTM - Classification report (Activo: ZS)



(e) GRU - Classification report (Activo: ZS)

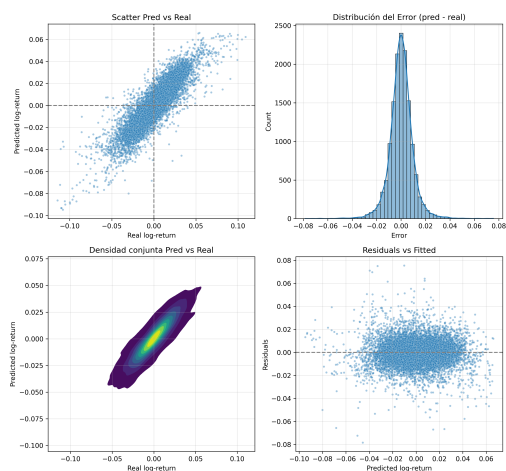


(b) LSTM - ROC y reliability (Activo: ZS)

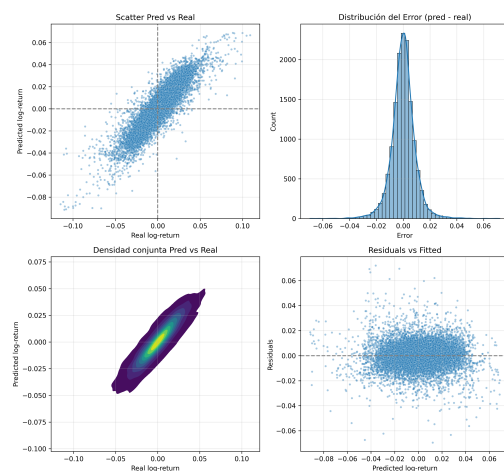


(f) GRU - ROC y reliability (Activo: ZS)

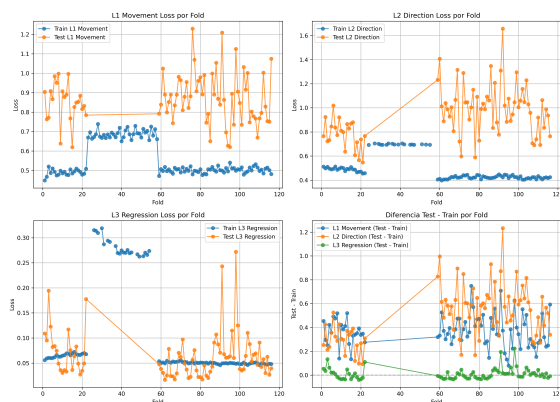
Figura 7.3: Comparativa de métricas de clasificación y curvas ROC para los modelos LSTM y GRU aplicados al activo ZS.



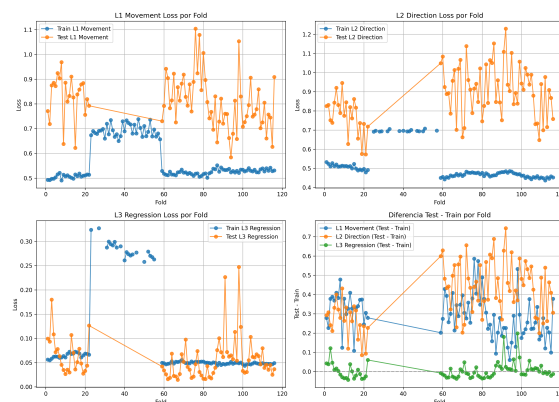
(c) LSTM - MAE y regresión (Activo: ZS)



(g) GRU - MAE y regresión (Activo: ZS)



(d) LSTM - Loss (Activo: ZS)



(h) GRU - Loss (Activo: ZS)

Figura 7.4: Comparativa de métricas de regresión y curvas de pérdida para los modelos LSTM y GRU aplicados al activo ZS.

C Repositorio de GitHub con el código

De igual modo que con los resultados del entrenamiento el total de las imágenes están subidas en GitHub.

<https://github.com/LijunZhou000/ML-Algorithmic-Trading/tree/main/TFG/img/Models>

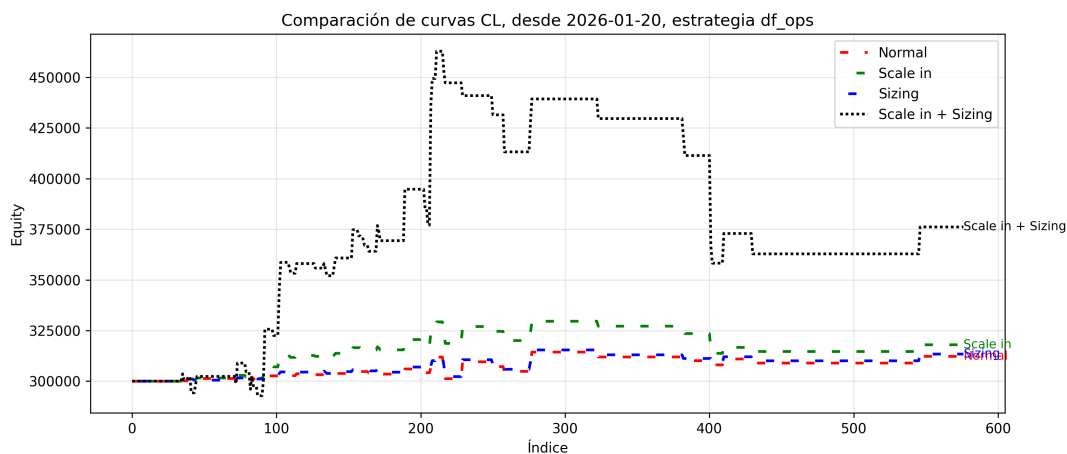


Figura 7.5: Ejemplo de curva de capital operando tanto LONG como SHORT para el activo CL

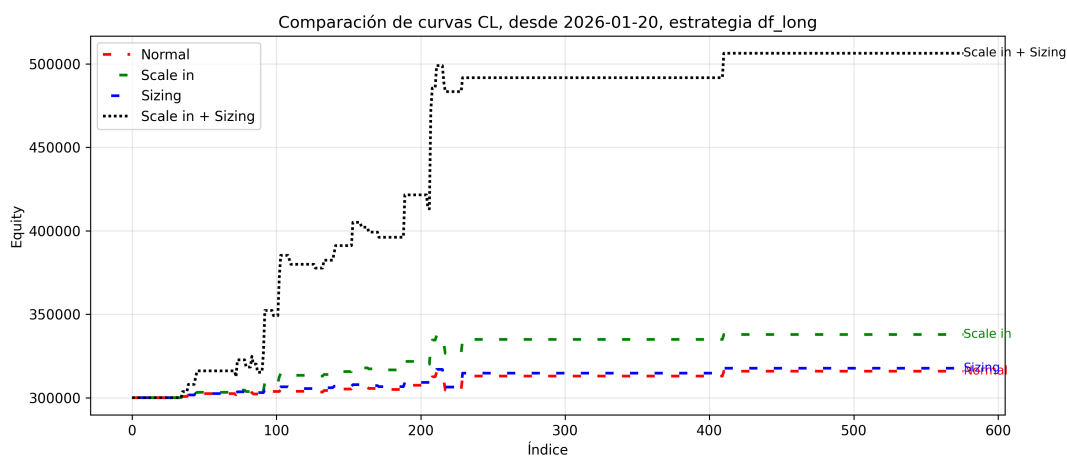


Figura 7.6: Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada por defecto para el activo CL

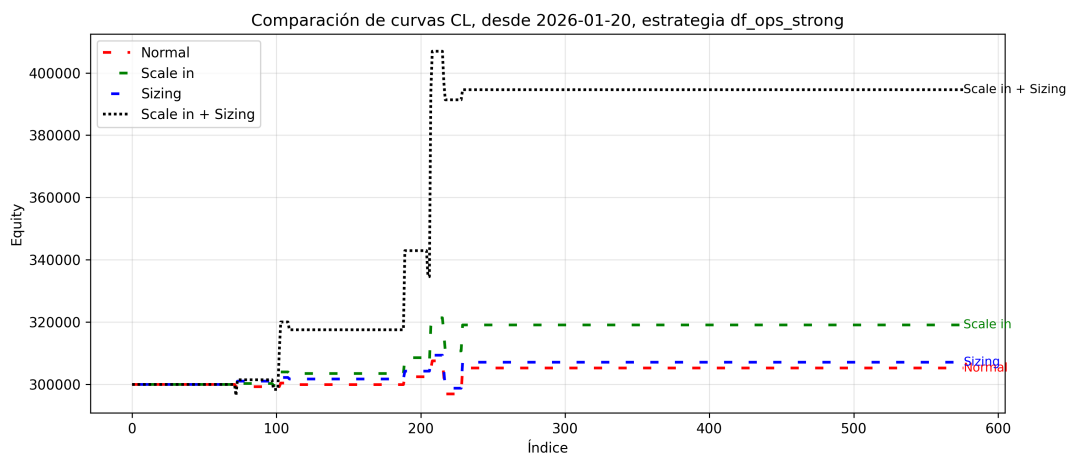


Figura 7.7: Ejemplo de curva de capital operando tanto LONG como SHORT con un umbral de entrada exigente para el activo CL

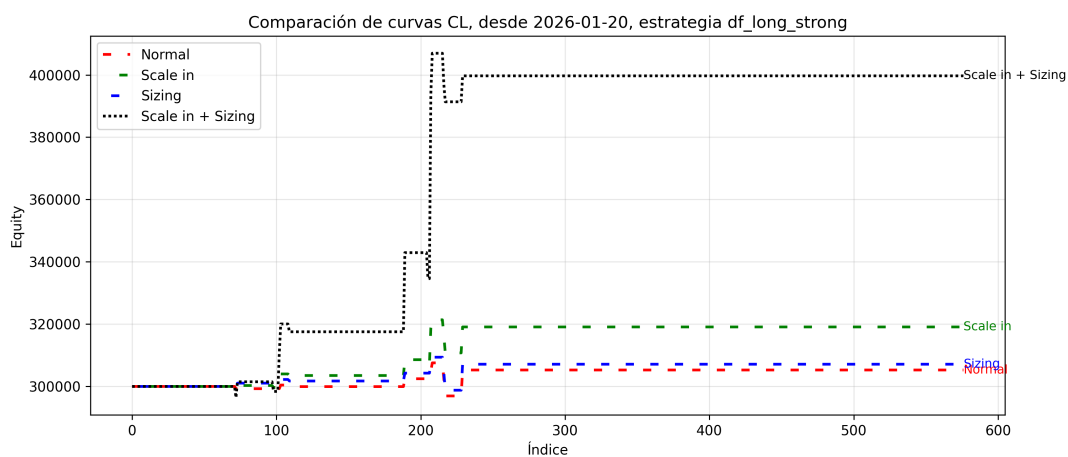


Figura 7.8: Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada exigente para el activo CL

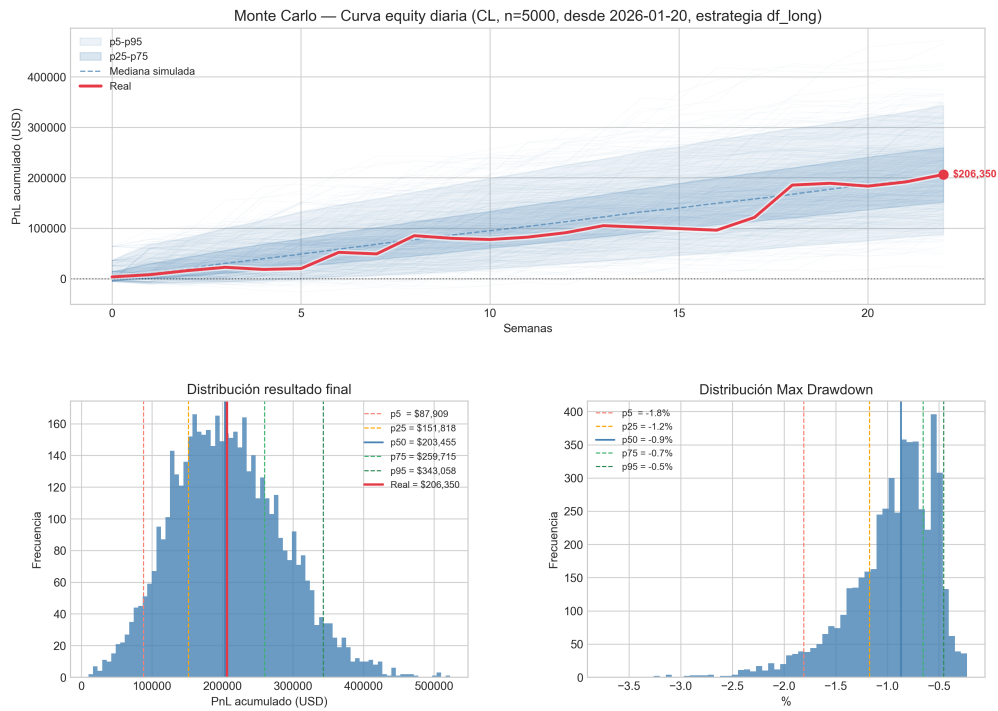


Figura 7.9: Ejemplo de Monte Carlo para la mejor estrategia del activo CL

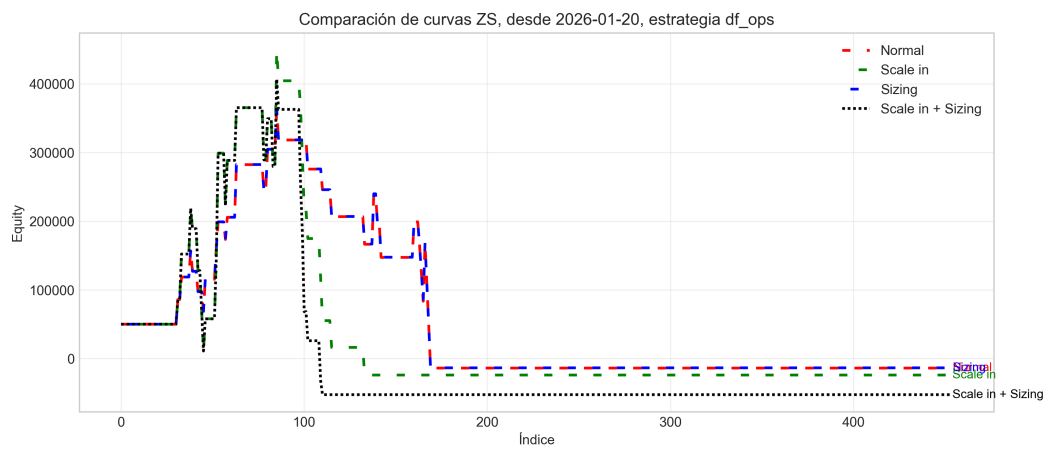


Figura 7.10: Ejemplo de curva de capital operando tanto LONG como SHORT para el activo ZS

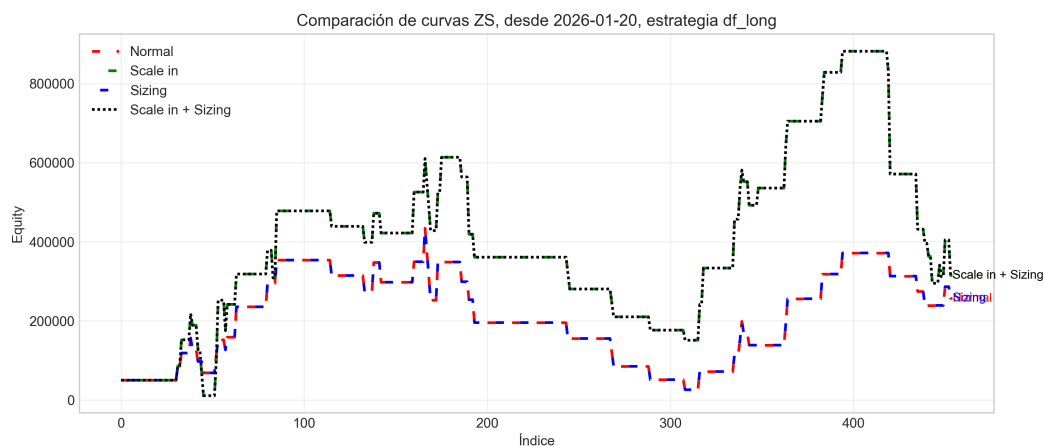


Figura 7.11: Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada por defecto para el activo ZS

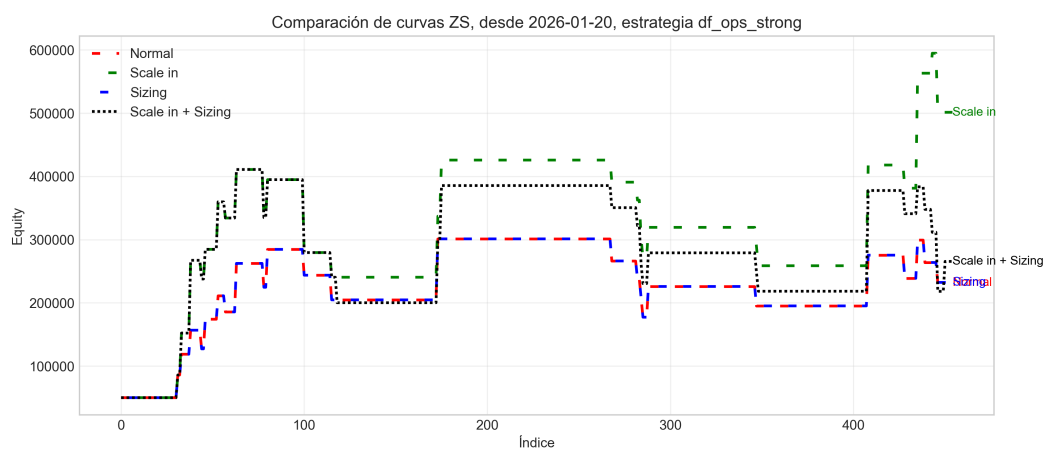


Figura 7.12: Ejemplo de curva de capital operando tanto LONG como SHORT con un umbral de entrada exigente para el activo ZS

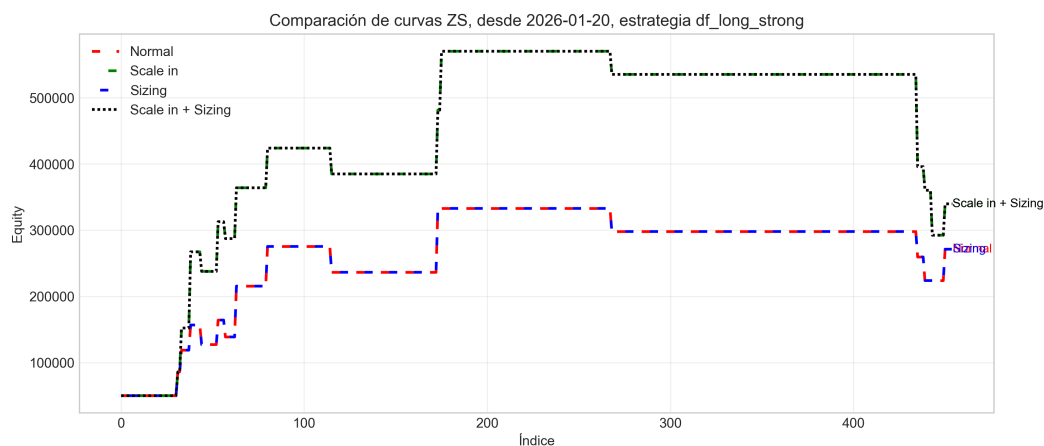


Figura 7.13: Ejemplo de curva de capital operando únicamente posiciones LONG con un umbral de entrada exigente para el activo ZS

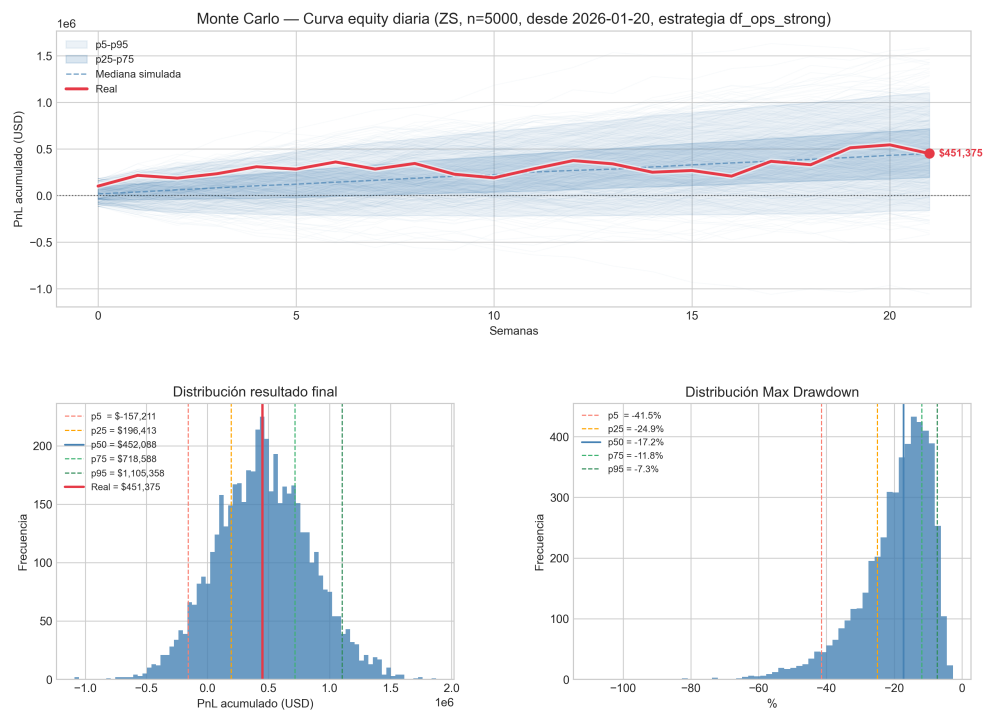


Figura 7.14: Ejemplo de Monte Carlo para la mejor estrategia del activo ZS